

Zelo+

Guia do produto



Imagem de referência — conceito visual do produto final (ilustrativo); não é o protótipo atual.

Dispenser inteligente de medicamentos com 3 compartimentos
ESP32 · servos · LCD I2C · Telegram · histórico de doses

Desenho técnico · funcionamento · montagem
componentes · custo de produção

Setembro de 2026

Este guia apresenta o dispenser inteligente de medicamentos Zelo+: a concepção do produto, o desenho técnico do gabinete, o funcionamento do programa (com o **trecho de código** de cada etapa; o arquivo completo está em [src/main.cpp](#)), a montagem para uso autônomo, a lista de componentes e a estimativa de custo de produção.

Sumário

• Parte A — Entendendo o projeto	3
• A1. O que é o Zelo+	3
• A2. Materiais e ligações	4
• A3. Ambiente de desenvolvimento	5
• A4. Cinco conceitos antes de começar	5
• A5. A página do Zelo+ (tela completa)	7
• Parte B — Desenho técnico do gabinete	9
• B1. Desenho técnico dimensional	9
• B2. Imagens do produto	11
• B3. Critérios de projeto	11
• B4. Onde fica cada componente	12
• Parte C — O código, passo a passo	13
• Passo 1 — Bibliotecas e pinos	13
• Passo 2 — Estruturas de dados	14
• Passo 3 — <code>setup()</code> : ligando tudo	15
• Passo 4 — Wi-Fi sem senha no código (captive portal)	16
• Passo 5 — Modo normal: relógio pela internet e rotas da página	16
• Passo 6 — Memória permanente (Preferences)	17
• Passo 7 — A página de cadastro	18
• Passo 8 — Salvando o cadastro	20
• Passo 9 — Movendo as portas (servos)	21
• Passo 10 — O coração: <code>loop()</code> e a máquina de estados	22
• Passo 11 — Conferindo os horários	23
• Passo 12 — O alarme tocando	23
• Passo 13 — O paciente aperta o botão	24
• Passo 14 — Abastecer, repor e esvaziar	25
• Passo 15 — Avisos pelo Telegram	26
• Passo 16 — Histórico de doses	27
• Passo 17 — Escrevendo no LCD	28
• Passo 18 — Modo de teste	29
• Passo 19 — Relógio DS3231 (opcional)	29
• Parte D — Juntando tudo	33
• D1. Linha do tempo de uma dose	33
• D2. Referência rápida	33
• Parte E — Uso autônomo, sem o laptop	35
• E1. Pré-requisitos e validação	35
• E2. Por que o laptop não é necessário	35
• E3. A solução: fonte 5 V/3 A + IP5306 + bateria 18650	35
• E4. Lista de compras	37
• E5. Montagem passo a passo	38
• E6. Teste de falta de energia	38
• E7. Autonomia da bateria	39
• E8. Falta de energia e de internet	39
• E9. Módulo de relógio DS3231	39
• E10. Atualizar o programa depois de montado	41
• Parte F — Lista completa de componentes, com preços	42
• F1. Eletrônica do dispenser	42
• F2. Energia autônoma	43
• F3. Montagem e ligações	44
• F4. Relógio sem internet (opcional)	44
• F5. Ferramentas necessárias (fora do custo)	45
• F6. Resumo do investimento	45
• Parte G — Custo de produção (estimativa para viabilidade)	47
• G1. Premissas e método	47
• G2. Volume de plástico do gabinete	48
• G3. Gabinete em impressão 3D	48
• G4. Gabinete em plástico injetado	49
• G5. Componentes eletrônicos no atacado	50
• G6. Custo por unidade em três escalas	51
• G7. Investimentos iniciais (uma vez só)	51
• G8. Preço de venda e viabilidade	52
• G9. Limitações da estimativa	52
• Referências	53

Parte A — Entendendo o projeto

A1. O que é o Zelo+

O Zelo+ é um **dispenser automático de medicamentos** para pacientes (em especial idosos). Ele tem **3 compartimentos**, cada um com um remédio diferente e com uma porta movida por um servo motor.



Imagem de referência — conceito visual do produto final (ilustrativo), com o botão à esquerda, o LCD no centro e a luz de alerta à direita.

Rotina do dia a dia:



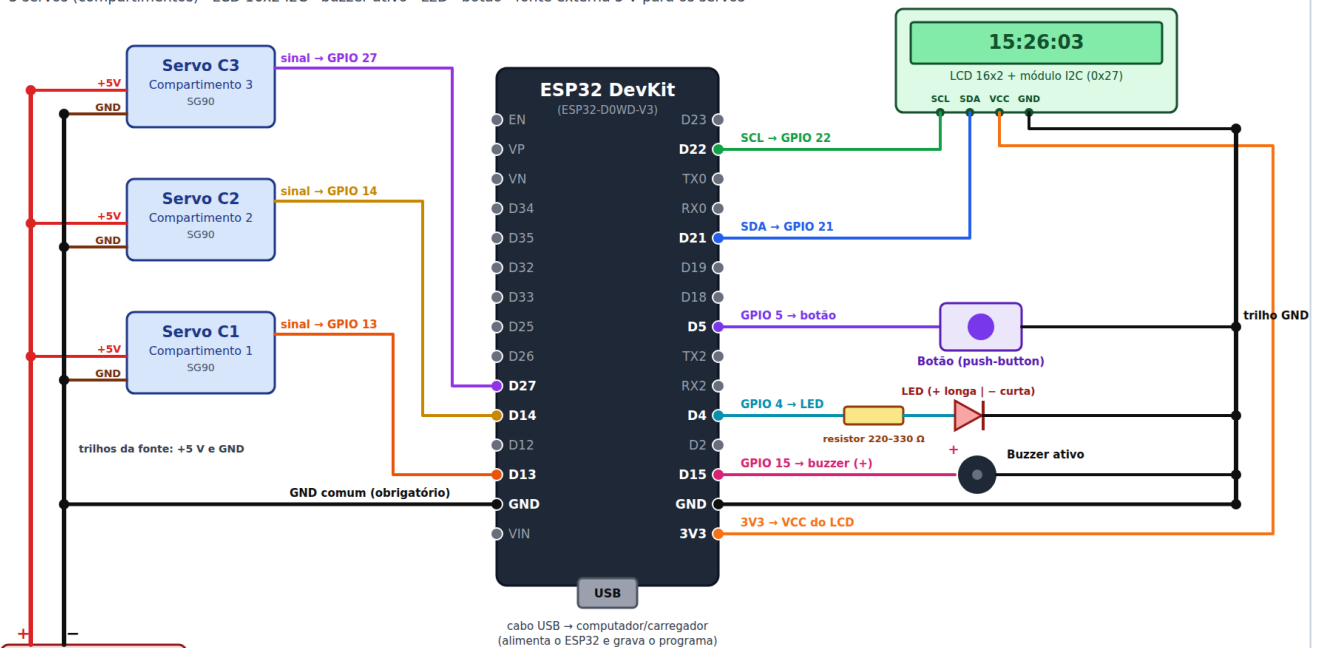
Rotina do dia a dia — cuidador, dispenser e paciente, do cadastro dos remédios aos avisos pelo Telegram.

A2. Materiais e ligações

Componente	Quantidade	Observação
ESP32 DevKit (30 pinos)	1	O "cérebro". Tem Wi-Fi embutido.
Servo motor SG90	3	Um por compartimento (porta).
LCD 16x2 com módulo I2C (PCF8574, endereço 0x27)	1	Mostra relógio e mensagens.
Buzzer ativo	1	Apita sozinho quando recebe energia.
LED + resistor 220-330 Ω	1	Pisca junto com o buzzer.
Botão (push-button)	1	O paciente aperta para abrir/fechar.
Fonte externa 5 V (≥ 2 A)	1	Alimenta os servos.
Relógio DS3231 + bateria LIR2032 (opcional)	1	Guarda a hora sem internet (Passo 19).
Protoboard e jumpers	—	Ligações.

Zelo+ — Diagrama de ligações (ESP32 DevKit 30 pinos)

3 servos (compartimentos) · LCD 16x2 I2C · buzzer ativo · LED · botão · fonte externa 5 V para os servos



Resumo das ligações

Servo C1 (compart. 1) — sinal
 Servo C2 (compart. 2) — sinal
 Servo C3 (compart. 3) — sinal
 LCD I2C — SDA / SCL
 Botão
 LED
 Buzzer ativo (+)
 Fonte 5 V (—)

GPIO 13
 GPIO 14
 GPIO 27
 GPIO 21 / GPIO 22
 GPIO 5
 GPIO 4 via resistor 220-330 Ω
 GPIO 15
 GND do ESP32

+5V e GND na fonte externa
 +5V e GND na fonte externa
 +5V e GND na fonte externa
 VCC → 3V3, GND → GND
 outra perna → GND (sem resistor: INPUT_PULLUP)
 perna curta → GND
 (—) → GND
 GND comum é obrigatório

Legenda

— +5 V (fonte externa)
 — GND (terra)
 — 3,3 V do ESP32
 — SDA (dados I2C)
 — SCL (clock I2C)
 ● = ligação (emenda)
 Fios que se cruzam SEM ● não estão ligados.
 Confira os nomes na serigrafia da sua placa:
 a ordem dos pinos varia entre modelos.

Diagrama de ligações da montagem de bancada. Versão vetorial, que pode ser ampliada sem perda de qualidade: [docs/ligacoes.svg](#).

Ligação	Pino do ESP32	Detalhe
Servo compartimento 1 (sinal)	GPIO 13	+5 V e GND na fonte externa
Servo compartimento 2 (sinal)	GPIO 14	+5 V e GND na fonte externa
Servo compartimento 3 (sinal)	GPIO 27	+5 V e GND na fonte externa
LCD — SDA / SCL	GPIO 21 / GPIO 22	VCC → 3V3, GND → GND
Relógio DS3231 — SDA / SCL (opcional)	GPIO 21 / GPIO 22	em paralelo com o LCD; VCC → 3V3, GND → GND (desenho na E3)
Botão	GPIO 5	outra perna → GND (sem resistor)
LED	GPIO 4	via resistor; perna curta → GND
Buzzer (+)	GPIO 15	(-) → GND
Fonte externa (-)	GND do ESP32	GND comum obrigatório

Por que uma fonte separada para os servos? Um servo puxa muita corrente no momento em que começa a girar. Se ele for ligado no 3V3 do ESP32, a tensão cai e a placa reinicia. Por isso os servos usam uma fonte de 5 V própria, e o **GND da fonte é ligado ao GND do ESP32**: sem essa referência comum, o sinal do pino não é entendido pelo servo.

Por que o GPIO 12 não foi usado? Ele é um pino de "configuração de inicialização" do ESP32; se estiver com algo ligado na hora de ligar a placa, ela pode não iniciar.

A3. Ambiente de desenvolvimento

- **VSCode** com a extensão **PlatformIO**.
- Placa `esp32dev`, framework **Arduino** (a mesma linguagem do Arduino, em C++).

O arquivo `platformio.ini` diz ao PlatformIO qual placa usar e quais bibliotecas baixar:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
upload_port = COM3
monitor_speed = 115200
board_build.partitions = huge_app.csv
lib_deps =
    marcoschwartz/LiquidCrystal_I2C^1.1.4
    madhephaestus/ESP32Servo^3.0.5
    bblanchon/ArduinoJson^7.0.4
    adafruit/RTClib^2.1.4
```

- `upload_port`: porta USB onde a placa aparece no Windows (varia de PC para PC).
- `monitor_speed`: velocidade do **Serial Monitor**, onde a placa escreve mensagens.
- `huge_app.csv`: reserva mais espaço para o programa (a parte de internet segura, HTTPS, deixa o firmware grande).
- `lib_deps`: bibliotecas do LCD, dos servos, de leitura de JSON (usada no Telegram) e do relógio DS3231 (`RTClib`).

Para gravar: ✓ (Build) → → (Upload) → 🖨️ (Serial Monitor), na barra azul do rodapé do VSCode.

A4. Cinco conceitos antes de começar

1. `setup()` e `loop()` — todo programa Arduino tem essas duas funções. O `setup()` roda **uma vez** quando a placa liga; o `loop()` roda **para sempre**, milhares de vezes por segundo.

2. **millis() em vez de delay()** — `delay(1000)` congela a placa por 1 segundo (ela não lê o botão nem atende a página). Por isso o Zelo+ quase sempre usa `millis()`, que devolve "há quantos milissegundos a placa está ligada", e compara tempos: `cpp if (millis() - ultimaAtualizacao < 1000) return;` (se ainda não passou 1 segundo desde a última atualização, sai e tenta de novo na próxima volta do `loop()`, sem travar nada).
3. **Máquina de estados** — o dispenser está sempre em **um** estado (aguardando, tocando, porta aberta, abastecendo). Em cada estado ele faz coisas diferentes e, quando algo acontece, **muda de estado**. Isso deixa o comportamento previsível.
4. **Máscara de bits** — para dizer "compartimentos 1 e 3", o código usa um número em que cada **bit** representa um compartimento:

```
compartimento:  3 2 1
bits:           1 0 1  = 5  →  compartimentos 1 e 3
```

Assim uma única variável (`uint8_t`) guarda qualquer combinação de compartimentos.

1. **Memória permanente (Preferences/NVS)** — a variável comum some quando a energia cai. A biblioteca `Preferences` grava dados na **memória flash** da placa, que não se apaga ao desligar (como um pen drive).

A5. A página do Zelo+ (tela completa)

É a página que o cuidador abre no celular, digitando o IP que aparece no LCD (ex.: 192.168.15.152). As imagens abaixo foram geradas **pelo próprio firmware** (a função `handleRoot()` do Passo 7) com um cadastro de exemplo completo.

① Falta configurar o Telegram

Zelo+

Os avisos pelo Telegram ainda não estão prontos (falta: Ana Silva). [Configurar agora](#)

Paulo Silva

Paciente

abrir ▼

Remédios

Cada remédio fica sempre no mesmo compartimento, identificado pela cor. Toque em um compartimento para ver ou mudar.

1

Losartana

Compartimento 1 (azul) - 08:00 - 20:00

abrir ▼

2

Metformina

Compartimento 2 (verde) - 12:00

abrir ▼

3

Sinvastatina

Compartimento 3 (roxo) - 21:00

abrir ▼

Contatos

Maria Souza

Cuidador(a)

abrir ▼

João Silva

Familiar

abrir ▼

Ana Silva

Familiar

abrir ▼

+ Cadastrar familiar

Salvar alterações

Repor remédio

Abrir compartimento para reposição

Histórico de doses

93% das doses foram tomadas nos últimos 7 dias

• 11 no horário • 2 com atraso • 1 sem acesso

1

Losartana: 86%

2

Metformina: 100%

3

Sinvastatina: 100%

Ver últimas doses (14)

abrir ▼

Trocar rede Wi-Fi

Avisos pelo Telegram

falta configurar

fechar ▲

O Zelo+ avisa o cuidador e os familiares pelo Telegram quando o paciente não pega o remédio.

Token do bot:
(salvo — deixe em branco para manter)

Buscar IDs do Telegram

Maria Souza ✓

123456789

João Silva ✓

987654321

Ana Silva (falta)

ID do Telegram, ex.: 123456789

Salvar

Como configurar (passo a passo)

abrir ▼

② Tudo configurado

Zelo+

Paulo Silva

Paciente

abrir ▼

Remédios

Cada remédio fica sempre no mesmo compartimento, identificado pela cor. Toque em um compartimento para ver ou mudar.

1

Losartana

Compartimento 1 (azul) - 08:00 - 20:00

abrir ▼

2

Metformina

Compartimento 2 (verde) - 12:00

abrir ▼

3

Sinvastatina

Compartimento 3 (roxo) - 21:00

abrir ▼

Contatos

Maria Souza

Cuidador(a)

abrir ▼

João Silva

Familiar

abrir ▼

Ana Silva

Familiar

abrir ▼

+ Cadastrar familiar

Salvar alterações

Repor remédio

Abrir compartimento para reposição

Histórico de doses

93% das doses foram tomadas nos últimos 7 dias

• 11 no horário • 2 com atraso • 1 sem acesso

1

Losartana: 86%

2

Metformina: 100%

3

Sinvastatina: 100%

Ver últimas doses (14)

abrir ▼

Trocar rede Wi-Fi

Avisos pelo Telegram

configurado ✓

abrir ▼

③ Blocos abertos

Zelo+

Paulo Silva

Paciente

abrir ▼

Remédios

Cada remédio fica sempre no mesmo compartimento, identificado pela cor. Toque em um compartimento para ver ou mudar.

1

Losartana

Compartimento 1 (azul) - 08:00 - 20:00

fechar ▲

20:00

Nome do remédio:
Losartana

Horários:
08:00
20:00

Para tirar um horário, apague-o.

+ Adicionar horário

Coloque Losartana no compartimento 1 (azul)

Esvaziar o compartimento 1

2

Metformina

Compartimento 2 (verde) - 12:00

abrir ▼

3

Sinvastatina

Compartimento 3 (roxo) - 21:00

abrir ▼

Contatos

Maria Souza

Cuidador(a)

abrir ▼

João Silva

Familiar

fechar ▲

Nome:
João Silva

Celular:
(11) 91234-5678

Remover este familiar

Ana Silva

Familiar

abrir ▼

Novo familiar

Familiar - opcional

fechar ▲

Nome:

Celular:
(11) 98765-4321

Cancelar

+ Cadastrar familiar

Salvar alterações

Repor remédio

Qual remédio você vai repor?

1

Losartana

2

Metformina

3

Sinvastatina

Continuar

Cancelar

- ① A familiar Ana ainda não tem ID do Telegram: aparece um **aviso amarelo no topo** e o bloco do Telegram (no fim da página) vem **aberto**, marcando quem falta.

- ② Com tudo configurado, a página fica curta: o Telegram vira uma linha "configurado ✓" no rodapé.
- ③ O que aparece ao tocar: compartimento 1 em edição (nome, horários, dica "coloque no compartimento 1 (azul)"), o familiar João aberto, o formulário de um **familiar novo** e a **reposição**, com a escolha do remédio pelas cores.

O visual segue três ideias, pensando em cuidadores e familiares com pouca familiaridade com tecnologia:

- **Letras e botões grandes**, com bom contraste.
- **Uma cor por compartimento** — 1 **azul**, 2 **verde**, 3 **roxo** — no cartão do remédio, na dica de abastecimento, na reposição e no histórico.
- **Menos informação na tela**: remédios, paciente, cuidador e familiares aparecem **recolhidos** (só a faixa com o nome); um toque abre os detalhes.

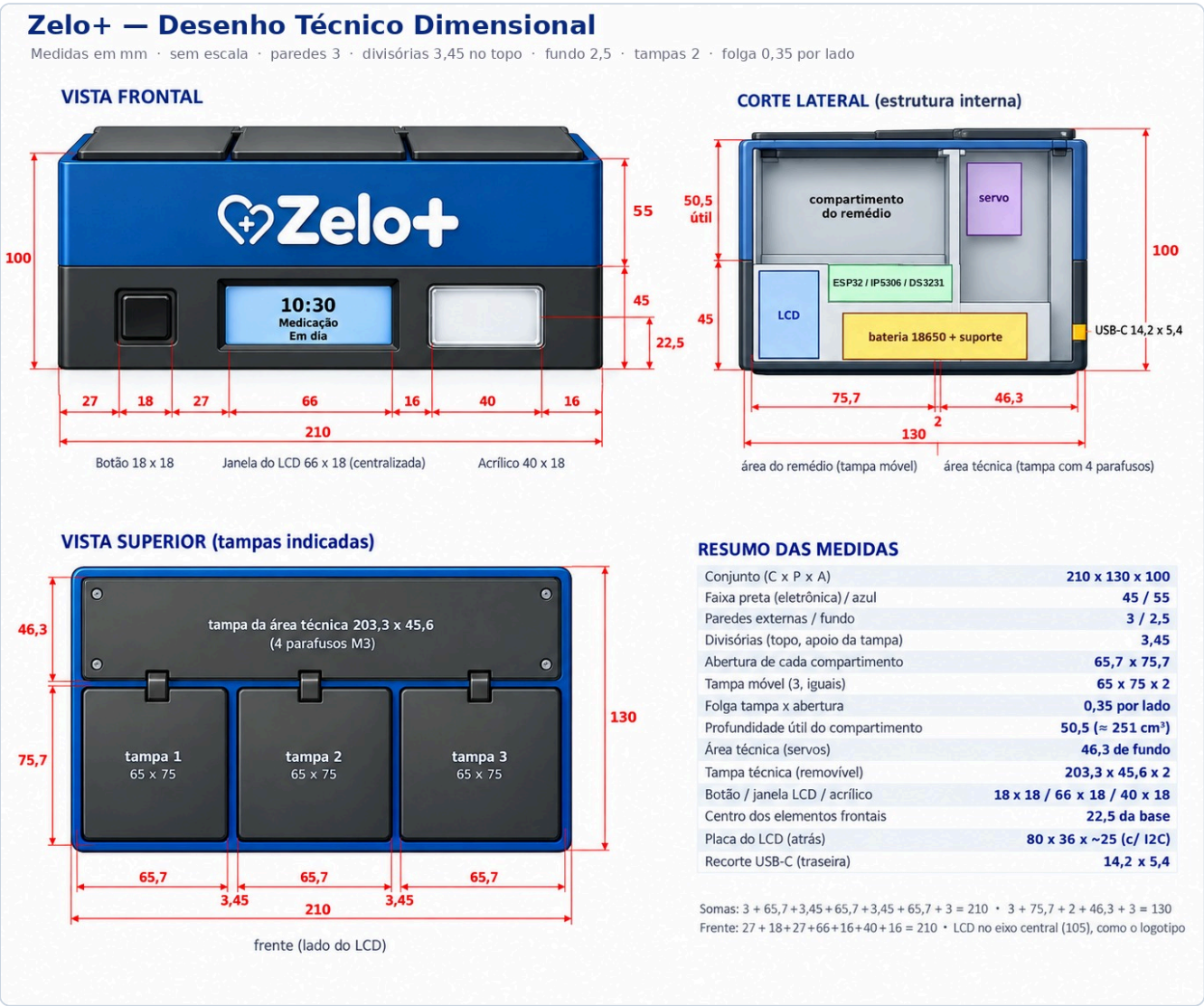
O que tem em cada bloco:

Bloco	Fechado mostra	Aberto mostra	Passo
Aviso amarelo (só se faltar algo)	Quem ainda não recebe avisos + "Configurar agora"	—	7
Paciente	Nome do paciente	Campo para editar o nome	7
Remédios 1, 2 e 3	Número, cor, remédio e horários	Nome, horários (até 6), dica de onde colocar, "Esvaziar"	7, 14
Cuidador(a)	Nome	Nome e celular	7, 8
Familiares	Nome de cada um	Nome, celular, "Remover"; "+ Cadastrar familiar" no fim	7
Salvar alterações	Botão azul grande	—	8
Repor remédio	Botão laranja	Escolha do remédio (por cor) e confirmação	14
Histórico de doses	% de adesão, contagem e adesão por remédio	"Ver últimas doses": tabela, planilha, apagar	16
Trocar rede Wi-Fi	Link (pede confirmação)	—	4
Relógio DS3231 (rodapé)	"conectado" ou "não instalado"	—	19
Avisos pelo Telegram	"configurado ✓" ou "falta configurar"	Token, "Buscar IDs", ID de cada pessoa, teste	15

Parte B — Desenho técnico do gabinete

O gabinete do Zelo+ mede **210 × 130 × 100 mm** (comprimento × profundidade × altura). Ele tem três compartimentos iguais na frente, uma área técnica na parte de trás (servos) e uma faixa de eletrônica na base, onde ficam o LCD, o botão e a luz de alerta. Os componentes citados nesta parte são explicados em detalhe nas Partes C (programa) e E (energia autônoma e relógio).

B1. Desenho técnico dimensional



Desenho técnico dimensional do gabinete — vista frontal, corte lateral, vista superior e resumo das medidas (em mm; vistas sem escala).

As vistas também estão em escala no arquivo vetorial [docs/gabinete.svg](#) (PNG).

Dimensões principais: 210 (C) × 130 (P) × 100 (A) mm.

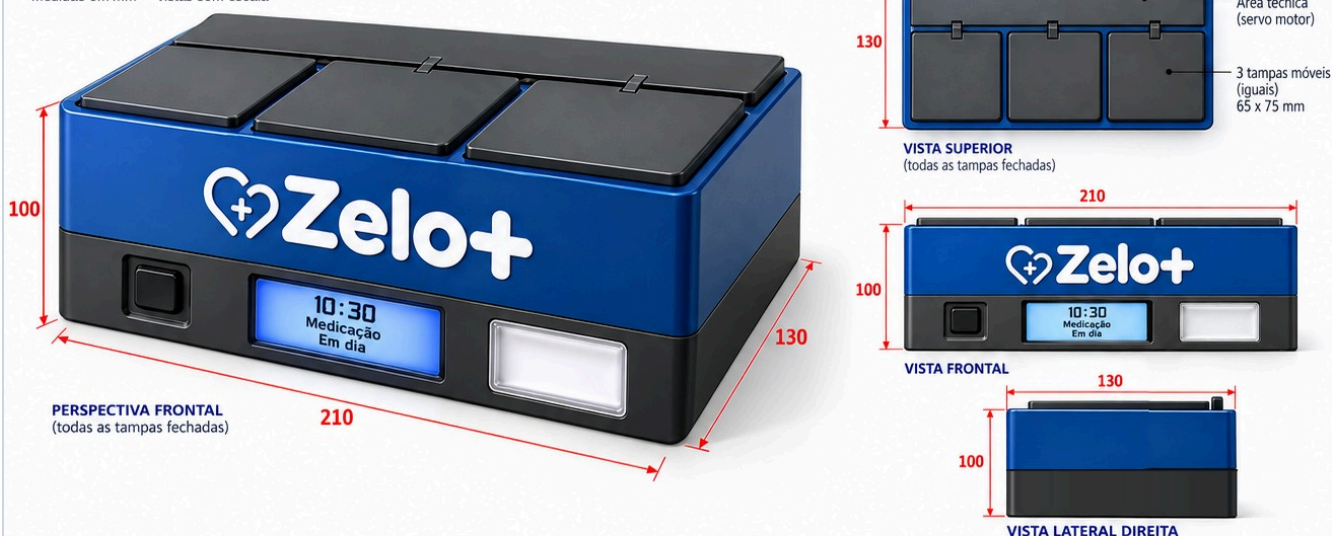
Direção	Composição (mm)	Soma
Largura (vista superior)	3 + 65,7 + 3,45 + 65,7 + 3,45 + 65,7 + 3	210 ✓
Profundidade	3 + 75,7 (compartimento) + 2 + 46,3 (área técnica) + 3	130 ✓
Altura	2,5 (fundo) + 42,5 (eletrônica) + 2,5 (piso) + 50,5 (compartimento) + 2 (tampa)	100 ✓
Frente	27 + 18 (botão) + 27 + 66 (janela do LCD) + 16 + 40 (acrílico) + 16	210 ✓

Elemento	Medida (mm)
Faixa preta (eletrônica) / faixa azul (compartimentos)	45 / 55
Abertura de cada compartimento	65,7 × 75,7
Tampa móvel (3, iguais)	65 × 75 × 2 (folga de 0,35 por lado)
Divisórias entre compartimentos	2 no corpo, 3,45 no topo (apoio das tampas)
Profundidade útil do compartimento	50,5 (≈ 251 cm ³ cada)
Tampa da área técnica (removível , 4 parafusos M3)	203,3 × 45,6 × 2
Botão / janela do LCD / acrílico	18 × 18 / 66 × 18 / 40 × 18, centros a 22,5 da base
Centros horizontais (botão / LCD / acrílico)	36 / 105 / 174
Placa do LCD (atrás do painel)	80 × 36 × ~25 (com módulo I2C)
Recorte do USB-C (parede traseira)	14,2 × 5,4, centro a ~15 da base

B2. Imagens do produto

Zelo+ | Desenho Técnico e Imagens

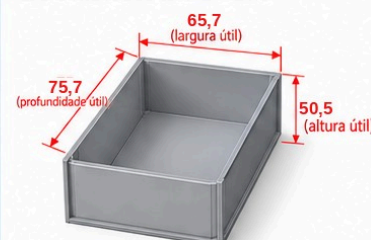
3 COMPARTIMENTOS IGUAIS | TAMPAS IGUAIS | ÁREA TÉCNICA POSTERIOR | LCD 16x2
Medidas em mm · vistas sem escala



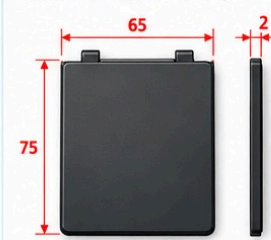
DETALHE DO COMPARTIMENTO ABERTO



MEDIDAS INTERNAS DO COMPARTIMENTO



VISTA DA TAMPA (cada uma das 3)



DETALHE DA DOBRADIÇA E ENCAIXE



Divisória entre compartimentos
(3,45 mm no topo)



Espessura da tampa
(2 mm)

Imagens do produto — perspectiva, vistas e detalhes do compartimento aberto, da dobradiça e do encaixe (medidas em mm; vistas sem escala).

B3. Critérios de projeto

- **Altura de 100 mm:**
- A proporção fica **210 : 130 : 100** ($\approx 2,1 : 1,3 : 1$), com a altura em $\sim 48\%$ da largura: perfil de caixa de mesa, estável e sem cara de "bloco".
- A faixa preta de 45 mm acomoda a placa do LCD (36 mm) com ~ 3 mm de margem acima e abaixo.
- Cada compartimento fica com **50,5 mm de profundidade útil** ($\sim 251 \text{ cm}^3$), cerca de 8 vezes o volume de um mês de comprimidos (60 unidades $\approx 30 \text{ cm}^3$). É raso o bastante para os dedos alcançarem o fundo.
- Opcional: um fundo em rampa de $10\text{--}15^\circ$ para a frente deixa os comprimidos mais perto da mão.
- **Frente simétrica:**
- A janela do LCD fica no **eixo central (105 mm)**, alinhada com o logotipo.

- O botão (centro em 36 mm) e o acrílico (centro em 174 mm) ficam cada um no meio do seu lado, **à mesma distância do centro (69 mm)**. Assim os três elementos ficam equilibrados, mesmo com larguras diferentes.
- **Folga das tampas:** para a tampa de 65 × 75 mm ter 0,35 mm de folga de cada lado, a abertura precisa medir 65,7 × 75,7 mm. Por isso as divisórias têm 3,45 mm no topo, onde as tampas se apoiam.
- **Manutenção:** a tampa da área técnica é **removível** (4 parafusos M3), com acesso aos servos, à bateria e ao USB do ESP32 para atualizar o programa (E10).
- **Energia:** o conector USB-C de painel fica na **parede traseira**, dentro da faixa de eletrônica.

B4. Onde fica cada componente

Componente	Local no gabinete
LCD 16x2 + módulo I2C	Faixa preta, atrás da janela central (placa 80 × 36, ~25 de profundidade)
Botão e LED (atrás do acrílico)	Faixa preta, nas laterais do LCD
ESP32, IP5306 e relógio DS3231	Faixa de eletrônica, sob os compartimentos
Bateria 18650 + suporte (77 × 21 × 20)	Faixa de eletrônica, deitada sob os compartimentos
3 servos SG90	Área técnica (46,3 mm de fundo), no alto, junto às dobradiças das tampas
Conector USB-C de painel	Parede traseira, ~15 mm acima da base
Capacitor 1000 µF	Junto aos servos, na área técnica

Parte C — O código, passo a passo

Passo 1 — Bibliotecas e pinos

O que faz: carrega as bibliotecas e dá nome aos pinos, para o resto do código falar em `BUTTON_PIN` em vez de "5".

```
#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include <WebServer.h>
#include <DNSServer.h>
#include <Preferences.h>
#include <time.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <ESP32Servo.h>
#include <RTCLib.h>
#include <esp_sntp.h>

#define BUZZER_PIN 15
#define LED_PIN 4
#define BUTTON_PIN 5
#define NUM_COMPARTIMENTOS 3
#define MAX_HORARIOS 6 // horarios por medicamento
#define MAX_FAMILIARES 5
#define MAX_HISTORICO 60
```

Biblioteca	Para quê
WiFi , WebServer , DNSServer	Conectar à internet e servir a página de cadastro
WiFiClientSecure , HTTPClient , ArduinoJson	Falar com o Telegram (HTTPS + JSON)
Preferences	Memória permanente
time.h	Relógio (hora certa pela internet)
Wire , LiquidCrystal_I2C	LCD pelo barramento I2C
ESP32Servo	Controlar os servos
RTCLib , esp_sntp.h	Relógio DS3231 e aviso de "hora acertada pela internet" (Passo 19)

Os pinos e ângulos de cada servo ficam em **vetores** (listas), um item por compartimento:

```
const int PINOS_SERVO[NUM_COMPARTIMENTOS] = {13, 14, 27};
// Angulos de cada porta (ajuste aqui se algum mecanismo precisar de outro valor).
const int ANGULO_FECHADO[NUM_COMPARTIMENTOS] = {0, 0, 0};
const int ANGULO_ABERTO[NUM_COMPARTIMENTOS] = {90, 90, 90};
```

`PINOS_SERVO[0]` é o compartimento **1** (em programação, a contagem começa em zero). Esse detalhe aparece no código inteiro: índice `c` = compartimento `c + 1`.

Por fim, os **objetos** que representam cada peça:

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo servos[NUM_COMPARTIMENTOS];
WebServer server(80);
DNSServer dnsServer;
Preferences preferences;
Preferences prefsCadastro;
Preferences prefsHistorico;
```

- `lcd(0x27, 16, 2)` : LCD no endereço I2C `0x27` , 16 colunas, 2 linhas.
- `server(80)` : servidor web na porta 80 (a porta padrão dos navegadores).
- Três `Preferences` , uma para cada "gaveta" da memória: Wi-Fi, cadastro e histórico.

Passo 2 — Estruturas de dados

O que faz: define como o programa guarda um **contato** e um **medicamento**.

```
struct Contato {
    String nome;
    String telefone;
    String chatId;
};

// Medicamento guardado num compartimento fixo (indice 0..2 = compartimento 1..3).
struct Medicamento {
    String nome;
    int totalHorarios = 0;
    int hora[MAX_HORARIOS];
    int minuto[MAX_HORARIOS];
    bool jaDisparado[MAX_HORARIOS];
};

String nomePaciente = "";
Medicamento medicamentos[NUM_COMPARTIMENTOS];
Contato cuidador;
Contato familiares[MAX_FAMILIARES];
```

- `struct` agrupa várias informações sob um nome só — como uma ficha.
- `medicamentos[3]` : a **posição no vetor é o compartimento**. O remédio em `medicamentos[1]` está sempre no compartimento 2. É isso que "prende" cada remédio ao seu compartimento.
- `jaDisparado[]` : impede que o mesmo horário toque duas vezes no mesmo minuto.
- `chatId` : número que o Telegram usa para identificar cada pessoa (Passo 15).

Os **estados** da máquina de estados:

```
enum Estado { AGUARDANDO, TOCANDO, PORTA_ABERTA_ESTADO, ABASTECENDO };
Estado estadoAtual = AGUARDANDO;
```

E as variáveis da **dose atual** (usam máscara de bits, conceito 4):

```
uint8_t doseMascara = 0;
String horarioDose = "";
String nomesDose = "";
unsigned long alarmeIniciadoEm = 0;
int nivelAvisoEnviado = 0; // 0 = nenhum, 1 = cuidador avisado, 2 = todos alertados
bool dosePendente = false; // alarme terminou sem acesso; botao ainda abre os compartimentos
```

Duas funções pequenas ajudam a trabalhar com isso:

```
bool temBit(uint8_t mascara, int i) {
    return (mascara >> i) & 1;
}

bool medicamentoAtivo(int c) {
    return medicamentos[c].nome != "" && medicamentos[c].totalHorarios > 0;
}
```

- `temBit(5, 0)` → verdadeiro (compartimento 1 está na máscara 101).
- `medicamentoAtivo(c)`: um compartimento só conta se tiver nome e horário.

Passo 3 — `setup()`: ligando tudo

O que faz: roda uma vez ao ligar. Configura pinos, posiciona as portas, liga o LCD, lê a memória e tenta conectar no Wi-Fi.

```
pinMode(BUZZER_PIN, OUTPUT);
pinMode(LED_PIN, OUTPUT);
pinMode(BUTTON_PIN, INPUT_PULLUP);
digitalWrite(BUZZER_PIN, LOW);
digitalWrite(LED_PIN, LOW);

for (int c = 0; c < NUM_COMPARTIMENTOS; c++) {
    servos[c].attach(PINOS_SERVO[c]);
    servos[c].write(ANGULO_FECHADO[c]);
}
```

- `OUTPUT`: o ESP32 **manda** energia no pino (buzzer, LED).
- `INPUT_PULLUP`: o pino fica "puxado" para 3,3 V por um resistor **interno**. Quando o botão é apertado, ele liga o pino ao GND e a leitura vira `LOW`. Por isso o botão não precisa de resistor externo, e "apertado" é `digitalRead(BUTTON_PIN) == LOW`.
- O `for` percorre os 3 servos, liga cada um no seu pino e fecha a porta.

Em seguida liga o LCD e **procura o relógio DS3231** (se ele não estiver instalado, o programa só anota isso e segue — detalhes no [Passo 19](#)):

```
lcd.init();
lcd.backlight();
lcd.print("Iniciando...");

iniciarRelogioRTC();
```

Depois, carrega a memória e decide o modo de Wi-Fi:

```
prefsCadastro.begin("cadastro", false);
carregarCadastro();
prefsHistorico.begin("historico", false);
carregarHistorico();

preferences.begin("wifi", false);
String ssidSalvo = preferences.getString("ssid", "");
String senhaSalva = preferences.getString("pass", "");
```

```
if (WiFi.status() == WL_CONNECTED) {
    iniciarModoNormal();
} else if (ssidSalvo != "" && horaDoRtc) {
    // Rede conhecida fora do ar, mas o DS3231 deu a hora: os alarmes funcionam
    // e o Wi-Fi continua sendo tentado em segundo plano (manterWiFi).
    iniciarModoNormal();
} else {
    iniciarModoConfig();
}
```

- Se existe rede salva e a conexão funciona → **modo normal** (Passo 5).
- Se a rede salva **não respondeu**, mas o **DS3231 deu a hora certa** → também modo normal, **sem internet**: os alarmes funcionam e o Wi-Fi é tentado de novo a cada 30 s (Passo 19).
- Nos outros casos (nenhuma rede salva, ou sem internet e sem relógio) → **modo de configuração** (Passo 4).

Passo 4 — Wi-Fi sem senha no código (captive portal)

O que faz: quando a placa não sabe em qual Wi-Fi entrar, ela **cria a sua própria rede** chamada `ZeloPlus-Config`. Ao conectar o celular nela, a página de configuração abre sozinha (igual ao Wi-Fi de aeroporto).

```
void iniciarModoConfig() {
    modoConfig = true;

    WiFi.mode(WIFI_AP_STA);
    WiFi.softAPConfig(apIP, apIP, IPAddress(255, 255, 255, 0));
    WiFi.softAP("ZeloPlus-Config");

    dnsServer.start(DNS_PORT, "*", apIP);
```

- `softAP` : transforma o ESP32 em um **ponto de acesso** (roteador).
- `dnsServer.start(..., "*", apIP)` : responde **qualquer** endereço digitado com o IP da placa (`192.168.4.1`). É esse "truque" que faz o celular abrir a página sozinho.

Quando o cuidador escolhe a rede e digita a senha, a placa testa e, se der certo, **grava** e reinicia:

```
if (WiFi.status() == WL_CONNECTED) {
    preferences.putString("ssid", ssid);
    preferences.putString("pass", senha);
```

```
delay(3000);
ESP.restart();
```

Da próxima vez que ligar, o `setup()` encontra a rede salva e vai direto para o modo normal. O link "Trocar rede Wi-Fi" da página apaga essa rede (`preferences.remove("ssid")`) e reinicia no modo de configuração.

Passo 5 — Modo normal: relógio pela internet e rotas da página

O que faz: acerta o relógio pela internet, mostra o IP no LCD e diz ao servidor qual função atende cada endereço da página.

```
const char* ntpServer = "pool.ntp.org";
const long gmtOffset_sec = -10800; // Brasília (GMT-3)
const int daylightOffset_sec = 0;
```

```
sntp_set_time_sync_notification_cb(aoSincronizarNTP);
configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
```

- **NTP** é um serviço da internet que informa a hora exata. `-10800` segundos = 3 horas a menos que o horário mundial (UTC) = horário de Brasília.
- A primeira linha pede ao NTP para **avisar** (chamando `aoSincronizarNTP`) toda vez que acertar a hora. É esse aviso que faz o programa gravar a hora no DS3231 (Passo 19).

Em seguida o LCD mostra o **IP** da página ou, se o dispenser ligou sem internet usando a hora do DS3231, o aviso Sem internet / Hora do relógio:

```
lcd.clear();
if (WiFi.status() == WL_CONNECTED) {
    lcd.print("IP do sistema:");
    lcd.setCursor(0, 1);
    lcd.print(WiFi.localIP());
    Serial.print("Acesse: http://");
    Serial.println(WiFi.localIP());
} else {
    lcd.print("Sem internet");
    lcd.setCursor(0, 1);
    lcd.print("Hora do relógio");
    Serial.println("Sem internet: alarmes pela hora do DS3231");
}
```

As **rotas** ligam um endereço a uma função:

```
server.on("/", handleRoot);
server.on("/salvar", HTTP_POST, handleSalvar);
server.on("/abrir-manual", HTTP_POST, handleAbrirManual);
server.on("/esvaziar", HTTP_POST, handleEsvaziar);
server.on("/testar-mensagens", HTTP_POST, handleTestarMensagens);
server.on("/telegram-ids", HTTP_POST, handleTelegramIds);
server.on("/historico.csv", handleHistoricoCSV);
server.on("/limpar-historico", HTTP_POST, handleLimparHistorico);
server.on("/trocar-wifi", handleTrocarWifi);
server.begin();
```

Endereço	O que acontece
/	Mostra a página de cadastro (Passo 7)
/salvar	Recebe o formulário e grava (Passo 8)
/abrir-manual	Abre um compartimento para reposição (Passo 14)
/esvaziar	Esvazia um compartimento (Passo 14)
/testar-mensagens , /telegram-ids	Telegram (Passo 15)
/historico.csv , /limpar-historico	Histórico (Passo 16)

Passo 6 — Memória permanente (Preferences)

O que faz: grava e lê o cadastro na memória flash, para ele sobreviver a quedas de energia.

Cada compartimento tem suas chaves: `m0_` (compartimento 1), `m1_`, `m2_`.

```
void salvarMedicamento(int c) {
    const Medicamento& m = medicamentos[c];
    String p = prefixoMedicamento(c);
    uint8_t horas[MAX_HORARIOS];
    uint8_t minutos[MAX_HORARIOS];
    for (int i = 0; i < m.totalHorarios; i++) {
        horas[i] = (uint8_t)m.hora[i];
        minutos[i] = (uint8_t)m.minuto[i];
    }
    prefsCadastro.putString((p + "_nome").c_str(), m.nome);
    prefsCadastro.putUChar((p + "_tot").c_str(), (uint8_t)m.totalHorarios);
    if (m.totalHorarios > 0) {
        prefsCadastro.putBytes((p + "_hr").c_str(), horas, m.totalHorarios);
        prefsCadastro.putBytes((p + "_mn").c_str(), minutos, m.totalHorarios);
    }
}
```

- `putString` grava texto; `putUChar` grava um número pequeno (0-255); `putBytes` grava uma lista de bytes (as horas e os minutos).
- Exemplo do que fica gravado para "Losartana às 08:00 e 20:00" no compartimento 1:
`m0_nome = "Losartana", m0_tot = 2, m0_hr = [8, 20], m0_mn = [0, 0]`.

Na leitura, o programa **confere** se os dados fazem sentido antes de usar:

```
int validos = 0;
for (int i = 0; i < total; i++) {
    if (horas[i] > 23 || minutos[i] > 59) continue;
    horaDestino[validos] = horas[i];
    minutoDestino[validos] = minutos[i];
    disparadoDestino[validos] = false;
    validos++;
}
return validos;
```

Migração: a versão anterior (um compartimento só) gravava o remédio em chaves diferentes. Na primeira vez que a versão nova liga, ela copia esse remédio para o compartimento 1 e apaga as chaves antigas:

```
void migrarCadastroAntigo() {
    if (prefsCadastro.isKey("m0_nome") || !prefsCadastro.isKey("remedio")) return;

    Medicamento& m = medicamentos[0];
    m.nome = prefsCadastro.getString("remedio", "");
    m.nome.trim();
    m.totalHorarios = lerHorarios("total", "horas", "minutos", m.hora, m.minuto, m.jaDisparado);
    salvarMedicamento(0);
}
```

- `trim()` tira espaços do começo e do fim do nome. Versões antigas podiam gravar "Losartana " (com espaço), e sem essa limpeza o sistema achava que o remédio tinha mudado e abria o compartimento sem necessidade.

Passo 7 — A página de cadastro

O que faz: quando o navegador abre o IP da placa, a função `handleRoot()` **monta o HTML** (o texto da página) e envia. O visual foi pensado para quem tem pouca familiaridade com tecnologia: **letras e botões grandes, uma cor por compartimento** e **blocos recolhidos** — só o essencial aparece; um toque abre os detalhes.

1) Uma cor por compartimento. Três listas guardam a cor de cada um (use etiquetas das mesmas cores na caixa do dispenser):

```
const char* COR_COMPARTIMENTO[NUM_COMPARTIMENTOS] = {"#2563eb", "#16a34a", "#9333ea"};
const char* FUNDO_COMPARTIMENTO[NUM_COMPARTIMENTOS] = {"#fff6ff", "#f0fdf4", "#faf5ff"};
const char* NOME_COR[NUM_COMPARTIMENTOS] = {"azul", "verde", "roxo"};
```

2) Blocos recolhidos. O HTML tem um elemento pronto para isso: `<details>` mostra só o `<summary>` (a "faixa") e esconde o resto até alguém tocar. Cada compartimento é um `<details>` com a sua cor; na faixa aparecem número, remédio e horários:

```
String html = "<details class='comp' style='--cor:" + String(COR_COMPARTIMENTO[c]) + ";--  
fundo:" +  
    String(FUNDO_COMPARTIMENTO[c]) + "' + (aberto ? " open" : "") + "><summary>";  
html += marcaCompartimento(c, 44);  
if (medicamentoAtivo(c)) {  
    html += "<span><b>" + escaparHTML(m.nome) + "</b><small>Compartimento " + n + " (" +  
NOME_COR[c] + ") &middot; " +  
        horariosTexto(m) + "</small></span>";  
} else {  
    html += "<span><b>Compartimento " + n + "</b><small>" + NOME_COR[c] + " &middot; vazio  
&middot; toque para cadastrar</small></span>";  
}
```

- `--cor` e `--fundo` são **variáveis de CSS**: o mesmo estilo (`.comp`) pinta a borda e o fundo de cada cartão com a cor do seu compartimento.
- `marcaCompartimento()` desenha a bolinha colorida com o número (ela também aparece na reposição e no histórico).
- O compartimento 1 só vem aberto no primeiro uso (nenhum remédio cadastrado).

Dentro do cartão, a **dica de onde colocar o remédio** repete número e cor:

```
html += "&#128230; Coloque <b>" + escaparHTML(m.nome) + "</b> no <b>compartimento " + n + " (" +  
+ NOME_COR[c] + ")</b></div>";
```

Resultado:  Coloque **Losartana** no **compartimento 1 (azul)**. A dica muda **enquanto o cuidador digita**, com um pequeno código JavaScript que roda no navegador:

```
html += "function atualizarDica(c) {";  
html += "    var nome = document.getElementById('m' + c + '_nome').value.trim();";  
html += "    var dica = document.getElementById('dica-' + c);";  
html += "    dica.style.display = nome ? 'block' : 'none';";
```

3) Pessoas recolhidas. Cuidador e familiares aparecem **só pelo nome**; tocando, abrem nome e celular. O familiar novo fica atrás do botão "+ Cadastrar familiar":

```
html += "<summary><span style='font-size:28px'>" + String(icone) + "</span><span><b>";  
html += contato.nome != "" ? escaparHTML(contato.nome) : String(familiar ? "Novo familiar" :  
"Cuidador(a)");
```

Cada familiar ocupa uma "vaga" (`fam0` a `fam4`). Ao cadastrar, o navegador procura a primeira vaga livre; ao remover, apaga o ID do Telegram daquela vaga — assim um familiar novo nunca herda o ID de quem saiu:

```
html += "    if (usados.indexOf('fam' + i) >= 0) continue;";  
html += "    var modelo = document.getElementById('modelo-  
familiar').innerHTML.split('famX').join('fam' + i);";
```

4) Telegram só quando precisa. O bloco do Telegram fica no **fim da página**, depois de "Trocar rede Wi-Fi". Ele só aparece **aberto** (e com um aviso no topo) enquanto faltar o token ou o ID de alguém — por exemplo, logo depois de cadastrar um familiar novo. Configurado, vira uma linha discreta "configurado ✓":

```
bool telegramPendente(String* faltando) {  
    bool pendente = (tokenTelegram == "");  
    if (cuidador.nome != "" && cuidador.chatId == "") {  
        pendente = true;
```

Os campos desse bloco ficam **fora** do formulário principal, mas são enviados junto graças ao atributo `form='cadastro'`:

```
html += "</label><input type='text' inputmode='numeric' form='cadastro' name='" + prefixo +
"_chat' id='chat-" + prefixo +
```

5) Troca de remédio num compartimento já usado — antes de enviar, o navegador compara o nome antigo (guardado em `data-original`) com o novo e pede confirmação:

```
html += "    if (antigo && novo && antigo.toLowerCase() != novo.toLowerCase()) {";
html += "        if (!confirm('O compartimento ' + (c + 1) + ' (' + CORES[c] + ') tinha ' +
antigo + '. Retire todos os comprimidos antigos antes de colocar ' + novo + '. Confirmar
troca?')) return false;";
```

Segurança: todo texto digitado passa por `escaparHTML()` antes de entrar na página. Assim, um nome como `D'Ávila` ou `<b>` não quebra o HTML.

```
cpp case '<': saida += "&lt;"; break;
```

Passo 8 — Salvando o cadastro

O que faz: `handleSalvar()` recebe o formulário, **valida** tudo e só então grava.

1. Cuidador é obrigatório:

```
if (novoCuidador.nome == "" || novoCuidador.telefone == "") {
    server.send(400, "text/plain; charset=utf-8", "O cuidador é obrigatório: preencha nome e
celular.");
    return;
}
```

1. Cada remédio precisa de horário, e ao menos um remédio precisa existir:

```
if (novos[c].nome != "" && novos[c].totalHorarios == 0) {
```

```
    if (ativos == 0) {
        server.send(400, "text/plain; charset=utf-8", "Cadastre ao menos um medicamento. Volte e
corrija.");
        return;
    }
}
```

1. Descobre quais compartimentos receberam remédio novo e os coloca numa fila de abastecimento (máscara de bits):

```
if (novo != "" && novo != antigo) filaAbastecimento |= (uint8_t)(1 << c);
if (novo == "") filaAbastecimento &= (uint8_t)~(1 << c);
medicamentos[c] = novos[c];
```

- `|= (1 << c)` **liga** o bit do compartimento `c` ("este precisa abrir").
- `&= ~(1 << c)` **desliga** o bit.
- Se só os horários mudaram, `novo == antigo` e nada entra na fila → a porta não abre.
- **Grava tudo** e devolve o navegador para a página:

```
salvarCadastro();

server.setHeader("Location", "/");
server.send(303);
```

A abertura das portas **não** acontece aqui: quem cuida disso é o `loop()` (Passo 10), um compartimento de cada vez.

Passo 9 — Movendo as portas (servos)

O que faz: abre e fecha as portas **devagar e sem trancos**. Cada movimento dura 2,5 s: a porta começa lenta, acelera no meio e freia ao chegar.

```
// Duracao de cada movimento de porta (abrir ou fechar), em milissegundos.
// Aumente para deixar as portas mais lentas.
const unsigned long TEMPO_MOVIMENTO_PORTA = 2500;

// Move a porta devagar e sem trancos: começa lento, acelera no meio e freia no
// fim (curva de cosseno). Um passo a cada 20 ms, o intervalo do sinal do servo.
void moverPorta(int c, int de, int para) {
    const int passos = TEMPO_MOVIMENTO_PORTA / 20;
    for (int i = 1; i <= passos; i++) {
        float t = (float)i / passos;           // 0 -> 1 ao longo do movimento
        float suave = (1 - cos(t * PI)) / 2;   // 0 -> 1, lento no inicio e no fim
        servos[c].write(de + (int)round((para - de) * suave));
        delay(20);
    }
}
```

- **Passos:** $2.500 \text{ ms} \div 20 \text{ ms} = \mathbf{125 \text{ posições}}$ por movimento. O servo recebe uma nova posição a cada pulso do seu sinal (a cada 20 ms).
- **Curva suave:** $(1 - \cos(t \cdot \pi)) / 2$ vai de 0 a 1 com início e fim lentos. Na abertura, a porta está em $\sim 9^\circ$ aos 0,5 s, em 45° na metade do tempo e chega a 90° aos 2,5 s. Isso evita o impacto da tampa no fim do curso e reduz o pico de corrente na partida do motor.
- **Ajuste:** para mudar a velocidade, basta alterar `TEMPO_MOVIMENTO_PORTA` (ex.: 3000 para 3 s).

```
void abrirCompartimento(int c) {
    moverPorta(c, ANGULO_FECHADO[c], ANGULO_ABERTO[c]);
}

void fecharCompartimento(int c) {
    moverPorta(c, ANGULO_ABERTO[c], ANGULO_FECHADO[c]);
}
```

Para abrir **vários** compartimentos com um único toque, sem que dois servos se movam juntos (o que sobrecarregaria a fonte), eles abrem **um depois do outro**:

```
// Abre os compartimentos marcados, um logo apos o outro.
void abrirCompartimentos(uint8_t mascara) {
    for (int c = 0; c < NUM_COMPARTIMENTOS; c++) {
        if (temBit(mascara, c)) abrirCompartimento(c);
    }
}
```

Como `abrirCompartimento()` só termina depois que a porta chegou em 90° , o próximo servo só começa quando o anterior parou.

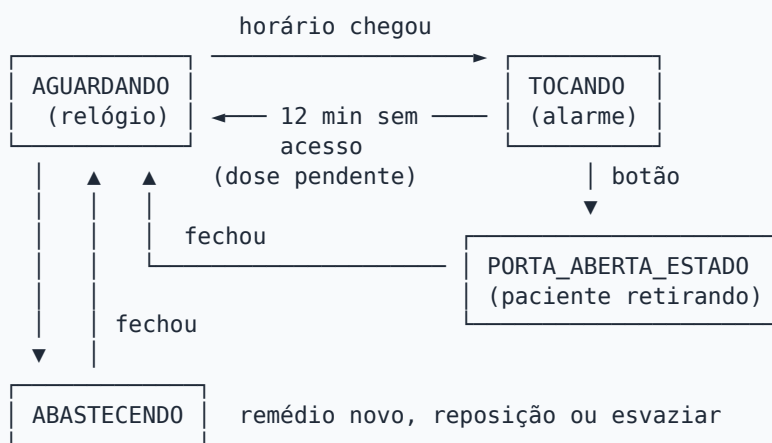
Passo 10 — O coração: `loop()` e a máquina de estados

O que faz: a cada volta, atende a página web, confere os horários e executa o que o **estado atual** manda.

```
void loop() {  
  if (modoConfig) {  
    dnsServer.processNextRequest();  
  }  
  server.handleClient();  
  
  if (modoConfig) {  
    return;  
  }  
  
  manterWiFi();  
  gravarHoraNoRTC();  
  verificarHorarios();  
  
  switch (estadoAtual) {
```

- `manterWiFi()` e `gravarHoraNoRTC()` cuidam do relógio e da reconexão (Passo 19). As duas voltam na hora quando não há nada a fazer, então não atrasam o alarme.

Mapa dos estados:



No estado **AGUARDANDO**, a ordem de prioridade é:

```
case AGUARDANDO:  
  if (dosePendente && digitalRead(BUTTON_PIN) == LOW) {  
    abrirParaPaciente();  
    break;  
  }  
  // Prioridade: alarme em espera; depois abastecimento de compartimento novo.  
  if (mascaraEmEspera != 0) {  
    uint8_t mascara = mascaraEmEspera;  
    mascaraEmEspera = 0;  
    iniciarAlarme(mascara, horarioEmEspera);  
    break;  
  }  
  if (filaAbastecimento != 0) {
```

1. Dose pendente + botão → abre para o paciente.
2. Algum horário esperando → começa o alarme.
3. Algum compartimento novo na fila → abre para abastecer.
4. Nada disso → mostra o relógio.

Passo 11 — Conferindo os horários

O que faz: uma vez por segundo, compara a hora atual com os horários de cada remédio. Funciona **em qualquer estado**.

```
for (int c = 0; c < NUM_COMPARTIMENTOS; c++) {
    if (!medicamentoAtivo(c)) continue;
    Medicamento& m = medicamentos[c];
    for (int i = 0; i < m.totalHorarios; i++) {
        if (m.jaDisparado[i] || timeinfo.tm_hour != m.hora[i] || timeinfo.tm_min != m.minuto[i])
            continue;
        m.jaDisparado[i] = true;
        if (mascaraEmEspera == 0) horarioEmEspera = formatarHorario(m.hora[i], m.minuto[i]);
        mascaraEmEspera |= (uint8_t)(1 << c);
    }
}
```

- Quando bate o horário, o compartimento entra na **máscara de espera**.
- **Dois remédios no mesmo horário** entram na mesma máscara → viram **um único alarme**.
- Se o dispenser estiver ocupado (ex.: cuidador repondo outro compartimento), o horário **fica esperando** e toca assim que ele voltar a **AGUARDANDO**.
- **jaDisparado** impede de tocar de novo no mesmo minuto; ele é "rearmado" quando o minuto muda:

```
if (timeinfo.tm_min != medicamentos[c].minuto[i]) medicamentos[c].jaDisparado[i] = false;
```

Passo 12 — O alarme tocando

O que faz: **iniciarAlarme()** prepara a dose e muda para **TOCANDO**.

```
void iniciarAlarme(uint8_t mascara, const String& horario) {
    estadoAtual = TOCANDO;
    doseMascara = mascara;
    horarioDose = horario;
    nomesDose = juntarNomes(mascara);
    alarmeIniciadoEm = millis();
    nivelAvisoEnviado = 0;
    dosePendente = false;
}
```

- **nomesDose** guarda o texto "Losartana e Metformina" para as mensagens.
- **alarmeIniciadoEm** marca o instante do início: todos os tempos são contados a partir dele.

Ciclo do som — 1 minuto tocando, 1 minuto em silêncio:

```
bool minutoDeSom = ((decorrido / CICLO_ALARME) % 2) == 0;
if (!minutoDeSom) {
    if (ledBuzzerLigado) desligarLedBuzzer();
    break;
}

if (millis() - ultimoToggleAlarme >= 150) {
    ultimoToggleAlarme = millis();
    ledBuzzerLigado = !ledBuzzerLigado;
    digitalWrite(LED_PIN, ledBuzzerLigado ? HIGH : LOW);
    digitalWrite(BUZZER_PIN, ledBuzzerLigado ? HIGH : LOW);
}
```

- **decorrido / CICLO_ALARME** = quantos minutos se passaram (0, 1, 2, ...).
- **% 2 == 0** → minutos pares (0, 2, 4...) tocam; ímpares ficam em silêncio.
- Dentro do minuto de som, LED e buzzer **invertem a cada 150 ms** (bip-bip-bip).

Avisos — aos 6 minutos para o cuidador, aos 12 para todos:

```
if (decorrido >= TEMPO_PRIMEIRO_AVISO && nivelAvisoEnviado == 0) {
    desligarLedBuzzer();
    escreverLinhaLCD(1, "Avisando cuidad.");
    avisarPrimeiroAtraso();
    nivelAvisoEnviado = 1;
}
```

```
if (decorrido >= TEMPO_ALERTA_FINAL) {
    desligarLedBuzzer();
    lcd.clear();
    escreverLinhaLCD(0, "Avisando familia");
    atualizarDose(DOSE_SEM_ACESSO, decorrido);
    alertarSemAcesso();
    nivelAvisoEnviado = 2;
}
```

```
dosePendente = true;
lcd.clear();
estadoAtual = AGUARDANDO;
```

- `nivelAvisoEnviado` evita mandar o mesmo aviso duas vezes.
- Depois dos 12 minutos o alarme **para**, mas a dose fica **pendente**: se o paciente aparecer mais tarde, o botão ainda abre os compartimentos.

Passo 13 — O paciente aperta o botão

O que faz: registra o acesso, desliga o alarme e **abre todos os compartimentos da dose**, um após o outro.

```
void abrirParaPaciente() {
    unsigned long decorrido = millis() - alarmeIniciadoEm;
    atualizarDose(decorrido < TEMPO_PRIMEIRO_AVISO ? DOSE_NO_HORARIO : DOSE_ATRASADA, decorrido);
    esquecerRegistrosDose();

    desligarLedBuzzer();
    dosePendente = false;

    lcd.clear();
    escreverLinhaLCD(0, "Retire: " + listaCompartimentos(doseMascara));
    escreverLinhaLCD(1, "Abrindo...");
    abrirCompartimentos(doseMascara);

    mascaraAberta = doseMascara;
    estadoAtual = PORTA_ABERTA_ESTADO;
    portaAbertaEm = millis();

    avisarAcessoAposAviso();
}
```

- Antes do 1º aviso → "tomada no horário"; depois → "tomada com atraso" (Passo 16).
- Se já tinha avisado alguém, `avisarAcessoAposAviso()` manda "Paulo acessou o dispenser às 15:27", para a família ficar tranquila.

Fechando — trava de 7 segundos: logo depois de abrir, o dedo do paciente ainda pode estar no botão. Para não fechar sem querer, o botão só vale depois de 7 s. Se ninguém apertar, fecha sozinho após 3 minutos:

```
bool travaLiberada = (millis() - portaAbertaEm >= TRAVA_BOTAO);
bool fechar = (travaLiberada && digitalRead(BUTTON_PIN) == LOW) ||
              (millis() - portaAbertaEm >= TEMPO_PORTA_ABERTA);

if (fechar) {
    lcd.clear();
    escreverLinhaLCD(0, "Fechando porta..");
    fecharCompartimentos(mascaraAberta);
    mascaraAberta = 0;
}
```

Passo 14 — Abastecer, repor e esvaziar

Os três casos usam a **mesma função**, que abre **um** compartimento para o cuidador e só muda o texto do LCD:

```
void iniciarAbastecimento(int c, ModoAbastecimento modo) {
    compartimentoAbastecendo = c;
    modoAbastecimento = modo;

    String titulo;
    if (modo == ABAST_REPOR) titulo = "Repor compart." + String(c + 1);
    else if (modo == ABAST_ESVAZIAR) titulo = "Esvaziar comp." + String(c + 1);
    else titulo = "Abast. compart." + String(c + 1);
}
```

a) Abastecimento guiado (remédio novo): o `loop()` pega o **primeiro** compartimento da fila, tira ele da fila e abre. Quando a porta fecha, o estado volta para `AGUARDANDO` e o próximo da fila abre — um de cada vez:

```
for (int c = 0; c < NUM_COMPARTIMENTOS; c++) {
    if (!temBit(filaAbastecimento, c)) continue;
    filaAbastecimento &= (uint8_t)~(1 << c);
    iniciarAbastecimento(c, ABAST_NOVO);
    break;
}
```

b) Reposição: na página, o cuidador **escolhe o remédio** (os botões de opção só mostram os cadastrados), confirma, e o navegador envia o número do compartimento:

```
html += " var corpo = new URLSearchParams(); corpo.append('c', reporEscolhido);";
html += " fetch('/abrir-manual', {method:'POST', body: corpo}).then(function(r){";
```

A placa confere se o pedido é válido e se está livre, e abre **só aquele**:

```
void handleAbrirManual() {
    int c = server.arg("c").toInt();
    if (!server.hasArg("c") || c < 0 || c >= NUM_COMPARTIMENTOS || !medicamentoAtivo(c)) {
        server.send(400, "text/plain", "Compartimento invalido");
        return;
    }
    if (!dispenserLivre()) {
        server.send(409, "text/plain", "Sistema ocupado no momento");
        return;
    }

    server.send(200, "text/plain", "OK");
    iniciarAbastecimento(c, ABAST_REPOR);
}
```

c) Esvaziar: apaga o remédio do compartimento e, se o cuidador pediu, abre para retirar as sobras:

```
medicamentos[c].nome = "";
medicamentos[c].totalHorarios = 0;
filaAbastecimento &= (uint8_t)~(1 << c);
salvarMedicamento(c);
```

Passo 15 — Avisos pelo Telegram

Por que Telegram? É gratuito, oficial e funciona com uma simples requisição pela internet. (O WhatsApp gratuito via CallMeBot ficou sem vagas; o WhatsApp oficial é pago e exige aprovação da Meta.)

Configuração (uma vez): o cuidador cria um bot no **BotFather**, cola o **token** na página; cada pessoa abre o bot e toca em **Iniciar**; o botão "Buscar IDs" mostra o **chatId** de cada uma.

Enviar uma mensagem é fazer um pedido HTTPS para a API do Telegram:

```
String url = "https://api.telegram.org/bot" + token + "/" + metodo;
if (!http.begin(cliente, url)) {
    Serial.println("Telegram: falha ao iniciar conexao");
    return -1;
}
http.addHeader("Content-Type", "application/x-www-form-urlencoded");
int codigo = http.POST(corpo);
```

```
String corpo = "chat_id=" + codificarURL(contato.chatId) + "&text=" + codificarURL(texto);
int codigo = chamarTelegram(tokenTelegram, "sendMessage", corpo);
```

- **codificarURL()** troca espaços e acentos por códigos (%20 , %C3%A3 ...), porque o endereço/corpo do pedido só aceita alguns caracteres.
- A resposta **200** significa "entregue ao Telegram".

As mensagens:

```
void avisarPrimeiroAtraso() {
    enviarParaCuidador("Zelo+: Paciente " + nomePaciente + " não acessou o medicamento das " +
        horarioDose + " horas (" + nomesDose + ").");
}

void alertarSemAcesso() {
    enviarParaTodos("Zelo+: Paciente " + nomePaciente + " não foi até o dispenser no horário das " +
        " +
        horarioDose + " (" + nomesDose + ").");
}
```

Momento	Quem recebe	Exemplo
6 min sem acesso	Cuidador	Zelo+: Paciente "Paulo" não acessou o medicamento das "15:26" horas (Losartana).
12 min sem acesso	Cuidador + familiares	Zelo+: Paciente "Paulo" não foi até o dispenser no horário das "15:26" (Losartana e Metformina).
Acesso depois de um aviso	Quem foi avisado	Zelo+: Paulo acessou o dispenser às 15:27 (dose de Losartana das 15:26).

Buscar IDs: a função `handleTelegramIds()` pede ao Telegram as últimas mensagens recebidas pelo bot (`getUpdates`) e usa a **ArduinoJson** para ler só o que interessa (id e nome de cada pessoa):

```
JsonDocument filtro;
JsonObject chatFiltro = filtro["result"][0]["message"]["chat"].to<JsonObject>();
chatFiltro["id"] = true;
chatFiltro["first_name"] = true;
chatFiltro["last_name"] = true;
chatFiltro["username"] = true;
```

O **filtro** economiza memória: a resposta do Telegram tem muitos campos, mas a placa guarda apenas esses quatro.

Passo 16 — Histórico de doses

O que faz: cada remédio de cada alarme vira um **registro** com data, situação e atraso. Serve para calcular a **adesão ao tratamento**.

```
struct RegistroDose {
    uint32_t inicio;           // horario do disparo (segundos desde 1970)
    uint16_t atrasoSeg;        // tempo ate o acesso
    uint8_t status;
    uint8_t compartimento;    // 0..2
    char remedio[20];          // nome do medicamento no momento da dose
};
```

```
enum StatusDose : uint8_t {
    DOSE_PENDENTE = 0,        // alarme tocando (ou placa reiniciou antes de concluir)
    DOSE_NO_HORARIO = 1,      // acesso antes do primeiro aviso ao cuidador
    DOSE_ATRASADA = 2,        // acesso depois do primeiro aviso (inclusive apos o alerta final)
    DOSE_SEM_ACESSO = 3      // alerta final enviado e ninguem acessou
};
```

Buffer circular: a placa guarda as **60 doses mais recentes**. Quando enche, a próxima sobrescreve a mais antiga — como uma fila que dá a volta:

```
int registrarInicioDose(int compartimento) {
    int idx = proximoHistorico;
```

```
    proximoHistorico = (proximoHistorico + 1) % MAX_HISTORICO;
    if (totalHistorico < MAX_HISTORICO) totalHistorico++;
    return idx;
}
```

- `% MAX_HISTORICO` (resto da divisão) faz a posição voltar a 0 depois da 59.

Para ler do mais recente para o mais antigo:

```
int indiceHistorico(int k) {
    return (proximoHistorico - 1 - k + 2 * MAX_HISTORICO) % MAX_HISTORICO;
}
```

Adesão = doses tomadas (no horário + com atraso) ÷ doses concluídas, nos últimos 7 dias. O arredondamento é feito somando metade do divisor:

```
int percentual(int parte, int total) {
    return (parte * 100 + total / 2) / total;
}
```

A página mostra a adesão geral e por remédio, a tabela das 20 doses mais recentes e o link **Baixar histórico (CSV)**, que abre no Excel/Google Planilhas:

```
String csv = "data;horario;medicamento;compartimento;situacao;atraso_minutos\r\n";
```

Passo 17 — Escrevendo no LCD

O problema: o LCD 16x2 não tem letras acentuadas. "Medicação" apareceria com símbolos estranhos. **A solução:** duas técnicas combinadas.

1. Caracteres especiais desenhados no LCD. O LCD permite criar até 8 caracteres próprios, cada um desenhado numa grade de 5 × 8 pontos (cada linha da grade é um número binário: 1 = ponto aceso). O Zelo+ cria o ç e o ã, que aparecem na tela de espera:

```
const uint8_t LCD_CEDILHA = 1;
const uint8_t LCD_A_TIL = 2;
uint8_t DESENHO_CEDILHA[8] = {0b00000, 0b01110, 0b10000, 0b10000, 0b10001, 0b01110, 0b00100, 0b01100};
uint8_t DESENHO_A_TIL[8] = {0b01101, 0b10010, 0b01110, 0b00001, 0b01111, 0b10001, 0b01111, 0b00000};
```

ç (código 1)	ã (código 2)
.	
.	
.	
.	
.	
.	
.	
.	

Os desenhos são gravados no LCD uma vez, ao ligar (no `setup()`):

```
lcd.createChar(LCD_CEDILHA, DESENHO_CEDILHA);
lcd.createChar(LCD_A_TIL, DESENHO_A_TIL);
```

2. As outras letras acentuadas perdem o acento. Em UTF-8 (o formato do texto), as letras acentuadas usam **2 bytes**, e o primeiro é sempre `0xC3`. O segundo byte diz qual é a letra. O "ç" (`0xA7`) e o "ã" (`0xA3`) viram os caracteres especiais; as outras viram a letra sem acento:

```
if (c == 0xC3 && i + 1 < texto.length()) {
    uint8_t original = (uint8_t)texto[++i];
    if (!maiusculas && original == 0xA7) { saida += (char)LCD_CEDILHA; continue; }
    if (!maiusculas && original == 0xA3) { saida += (char)LCD_A_TIL; continue; }
    uint8_t d = original | 0x20; // 0x80-0x9F (maiusculas) -> minusculas
    char base = '?';
    if (d >= 0xA0 && d <= 0xA5) base = 'A';
    else if (d == 0xA7) base = 'C';
    else if (d >= 0xA8 && d <= 0xAB) base = 'E';
}
```

Os **nomes dos remédios** são mostrados em **maiúsculas** (parâmetro `maiusculas`), o que facilita a leitura: "Losartana Potássica" → "LOSARTANA POTASS" (cortado em 16 colunas).

E uma função que **sempre preenche a linha inteira**, apagando restos de textos anteriores:

```
void escreverLinhaLCD(int linha, const String& texto) {
    String t = textoLCD(texto);
    while (t.length() < 16) t += ' ';
    lcd.setCursor(0, linha);
    lcd.print(t);
}
```

Na **tela de espera**, a linha 1 mostra a hora centralizada e a linha 2, a situação das doses:

```
// Linha 1: hora e minuto, centralizados.
char horaBuffer[6];
strftime(horaBuffer, sizeof(horaBuffer), "%H:%M", &timeinfo);
escreverLinhaLCD(0, String("      ") + horaBuffer);

int ativos = totalMedicamentosAtivos();
if (dosePendente) {
    escreverLinhaLCD(1, "Dose pendente!");
} else if (ativos > 0) {
    escreverLinhaLCD(1, "Medicação em dia");
} else {
    escreverLinhaLCD(1, "Sem remédio");
}
```

Situação	Linha 1	Linha 2
Aguardando	10:30 (centralizado)	Medicação em dia
Alarme	Hora do remedio!	LOSARTANA (C1) (alterna a cada 2 s)
Retirada	Retire: C1 C2	Fecha em: 170s
Abastecer / Repor	Abast. compart.2 / Repor compart.2	METFORMINA
Dose pendente	15:40 (centralizado)	Dose pendente!
Nenhum remédio cadastrado	hora (centralizada)	Sem remedio

Passo 18 — Modo de teste

Esperar 12 minutos a cada teste seria lento. Uma única linha troca todos os tempos:

```
#define MODO_TESTE 1

#if MODO_TESTE
const unsigned long CICLO_ALARME = 5UL * 1000UL;
const unsigned long TEMPO_PRIMEIRO_AVISO = 30UL * 1000UL;
const unsigned long TEMPO_ALERTA_FINAL = 60UL * 1000UL;
#else
const unsigned long CICLO_ALARME = 60UL * 1000UL;
const unsigned long TEMPO_PRIMEIRO_AVISO = 6UL * 60UL * 1000UL;
const unsigned long TEMPO_ALERTA_FINAL = 12UL * 60UL * 1000UL;
#endif
```

- `#if` é resolvido **na compilação**: só um dos blocos vai para a placa.
- Com `MODO_TESTE 1`: aviso aos 30 s, alerta aos 60 s, som 5 s / silêncio 5 s.
- **Antes do uso real**, trocar para `#define MODO_TESTE 0` e gravar de novo.
- O `UL` ("unsigned long") evita que a conta estoure: `12 * 60 * 1000` passa do limite de um `int` em algumas placas.

Passo 19 — Relógio DS3231 (opcional)

O que faz: se o **módulo de relógio DS3231** estiver ligado, o dispenser sabe a hora **mesmo sem internet**. Sem o módulo, nada muda: a hora continua vindo da internet (Passo 5). O **mesmo programa** serve para os dois casos, então o módulo pode ser ligado depois, sem gravar o programa de novo.

Por que existe: o relógio interno do ESP32 **zera quando a placa desliga**. Ao religar, ele só volta a saber a hora quando a internet responde. Se a energia voltar com o roteador ainda

desligado, os alarmes não tocariam. O DS3231 tem **bateria própria** (LIR2032) e continua contando o tempo com o dispenser desligado.

Ligação: no mesmo barramento I2C do LCD, em paralelo — SDA → GPIO 21, SCL → GPIO 22, VCC → 3V3, GND → GND (diagrama na [E3](#); detalhes do módulo na [E9](#)). O LCD usa o endereço 0x27 e o relógio 0x68, por isso os dois dividem os mesmos fios sem conflito.

1. As variáveis do relógio

```
// ----- RELOGIO DS3231 (opcional) -----  
// Modulo de relógio ligado no I2C, junto com o LCD (endereço 0x68). Guarda a hora  
// certa mesmo sem internet e sem energia. Sem o modulo, tudo funciona como antes,  
// com a hora vinda da internet (NTP).  
RTC_DS3231 rtc;  
bool rtcPresente = false;           // o modulo respondeu no I2C ao ligar  
bool horaDoRtc = false;            // a hora atual veio do DS3231 ao ligar  
volatile bool ntpSincronizou = false; // avisado pelo NTP a cada acerto de hora  
const uint32_t HORA_MINIMA_VALIDA = 1704067200UL; // 01/01/2024: antes disso, hora invalida
```

- `rtc` é o objeto que conversa com o módulo (biblioteca `RTClib`, da Adafruit).
- `HORA_MINIMA_VALIDA` é 1º de janeiro de 2024, contado em **segundos desde 1970** (o jeito que computadores guardam datas). Uma hora anterior a isso só pode estar errada — por exemplo, um módulo novo que nunca foi acertado.
- `volatile` avisa o compilador de que a variável pode mudar "por fora" do `loop()`: quem a muda é o NTP, que roda em paralelo.

2. Ao ligar: procurar o módulo e usar a hora dele

```
void iniciarRelogioRTC() {  
    rtcPresente = rtc.begin();  
    if (!rtcPresente) {  
        Serial.println("DS3231 nao encontrado - usando so a internet");  
        return;  
    }  
    if (rtc.lostPower()) {  
        Serial.println("DS3231 sem hora valida - aguardando a internet");  
        return;  
    }  
    uint32_t segundos = rtc.now().unixtime();  
    if (segundos < HORA_MINIMA_VALIDA) {  
        Serial.println("DS3231 com data antiga - aguardando a internet");  
        return;  
    }  
    struct timeval agora = { (time_t)segundos, 0 };  
    settimeofday(&agora, nullptr);  
    horaDoRtc = true;  
    Serial.println("DS3231: hora carregada do relógio");  
}
```

- `rtc.begin()` "chama" o endereço 0x68. **Sem o módulo, ninguém responde** e a função devolve `false` em uma fração de segundo: o programa anota e segue.
- `rtc.lostPower()` avisa se o módulo **perdeu a hora** (nunca foi acertado ou ficou sem bateria). Nesse caso o programa não confia nele e espera a internet.
- `settimeofday()` acerta o **relógio interno do ESP32** com a hora do módulo. Daí em diante, `getLocalTime()` (usado no relógio do LCD e na conferência dos horários) funciona normalmente, com ou sem internet.

3. Com internet: manter o módulo sempre certo

```
// Chamada pelo NTP (fora do loop) sempre que a hora e acertada pela internet.  
void aoSincronizarNTP(struct timeval* tv) {  
    ntpSincronizou = true;  
}
```

```
void gravarHoraNoRTC() {
    if (!ntpSincronizou) return;
    ntpSincronizou = false;
    if (!rtcPresente) return;
    time_t agora = time(nullptr);
    if ((uint32_t)agora < HORA_MINIMA_VALIDA) return;
    rtc.adjust(DateTime((uint32_t)agora));
    Serial.println("DS3231: hora gravada (vinda da internet)");
}
```

- O NTP acerta a hora ao conectar e depois **a cada hora**, mais ou menos. A cada acerto, `aoSincronizarNTP` só levanta uma "bandeira".
- Quem grava no módulo é `gravarHoraNoRTC()`, chamada pelo `loop()` (Passo 10). Assim a gravação nunca acontece no meio de uma escrita no LCD, que usa os mesmos fios.
- `rtc.adjust()` grava a hora e **apaga o aviso de hora perdida**: no próximo religamento, `lostPower()` já responde que a hora é confiável.
- O módulo guarda a hora **mundial (UTC)**; o fuso de Brasília é aplicado pelo `configTime()` na hora de mostrar. Por isso a hora não "anda" 3 horas ao religar.

4. Sem internet: continuar tentando o Wi-Fi

```
void manterWiFi() {
    static unsigned long ultimaTentativa = 0;
    static bool estavaConectado = true;
    bool conectado = (WiFi.status() == WL_CONNECTED);
    if (conectado && !estavaConectado) {
        Serial.print("Wi-Fi reconectado. Acesse: http://");
        Serial.println(WiFi.localIP());
    }
    estavaConectado = conectado;
    if (conectado || millis() - ultimaTentativa < 30000) return;
    ultimaTentativa = millis();
    WiFi.reconnect();
}
```

- Sem internet, tenta a rede salva **a cada 30 s**, sem travar o programa (o alarme continua funcionando enquanto isso).
- Quando conecta, a página volta a abrir **no mesmo IP**, o Telegram volta a funcionar e o NTP acerta a hora (e o DS3231 é regrado).

5. Na página: o relógio foi encontrado?

No rodapé, abaixo de "Trocar rede Wi-Fi", aparece o estado do módulo:

```
html += "<p class='rodape' style='font-size:15px;color:#6b7280;margin-top:0'>&#128339; Relógio  
DS3231: ";
html += rtcPresente ? "conectado" : "não instalado (hora pela internet)";
html += "</p>";
```

Resumo: o que acontece ao ligar

Situação ao ligar	O que o programa faz
Sem o módulo	Segue como antes: hora pela internet. Página: "não instalado".
Módulo novo (nunca acertado)	Espera a internet; no primeiro acerto, grava a hora no módulo .
Módulo com hora, com internet	Usa a hora do módulo na hora; o NTP confere e regrava a cada acerto.
Módulo com hora, sem internet	Modo normal sem internet : LCD Sem internet / Hora do relógio; alarmes funcionam ; Wi-Fi tentado a cada 30 s.
Nenhuma rede cadastrada	Modo de configuração (Passo 4), como antes.

As mensagens do Serial Monitor de cada situação estão na [E9](#).

Instalar o módulo depois: desligue o dispenser da energia, ligue os 4 fios e religue. Não é preciso gravar o programa de novo. O módulo só é procurado **ao ligar**; por isso o religamento é necessário.

A biblioteca `RTCLib` está no `platformio.ini` (`adafruit/RTCLib^2.1.4`). O PlatformIO baixa sozinho no próximo **Build**.

Parte D — Juntando tudo

D1. Linha do tempo de uma dose

Exemplo: **Losartana (compartimento 1)** e **Metformina (compartimento 2)** às **15:26**, modo real.

Hora	O que acontece	Onde no código
15:26:00	<code>verificarHorarios()</code> coloca C1 e C2 na máscara de espera	Passo 11
15:26:00	<code>loop()</code> em <code>AGUARDANDO</code> chama <code>iniciarAlarme(0b011, "15:26")</code> ; cria 2 registros "tocando"	Passos 10, 12, 16
15:26–15:27	Buzzer e LED bip-bip; LCD alterna <code>LOSARTANA (C1)</code> / <code>METFORMINA (C2)</code>	Passos 12, 17
15:27–15:28	Silêncio (botão continua valendo)	Passo 12
15:32	Sem acesso → Telegram para o cuidador	Passos 12, 15
15:33	Paciente aperta o botão → registros "com atraso (7 min)"; abre C1, depois C2	Passo 13
15:33	Telegram "Paulo acessou o dispenser às 15:33" para o cuidador	Passo 15
15:33:07	Trava liberada; LCD <code>Fecha em: 173s</code>	Passo 13
15:34	Paciente aperta → fecha C1, depois C2 → <code>AGUARDANDO</code>	Passo 13

Se o paciente **não** aparecer até 15:38 (12 min): alerta para todos, registros "sem acesso", LCD `Dose pendente!`, e o botão ainda abre os dois compartimentos depois.

D2. Referência rápida

Tempos

Constante	Modo real	Modo teste
<code>CICLO_ALARME</code>	1 min	5 s
<code>TEMPO_PRIMEIRO_AVISO</code>	6 min	30 s
<code>TEMPO_ALERTA_FINAL</code>	12 min	60 s
<code>TRAVA_BOTAO</code>	7 s	7 s
<code>TEMPO_PORTA_ABERTA</code> / <code>TEMPO_ABASTECIMENTO</code>	3 min	3 min
Nova tentativa de Wi-Fi (sem internet)	30 s	30 s

Endereços I2C (mesmos fios: GPIO 21 = SDA, GPIO 22 = SCL)

Peça	Endereço
LCD 16x2 (módulo PCF8574)	<code>0x27</code>
Relógio DS3231 (<i>opcional</i>)	<code>0x68</code>

Memória (Preferences)

Gaveta (namespace)	Chaves
wifi	ssid , pass
cadastro	nome (paciente), m0_* / m1_* / m2_* (remédios), cuid_* , fam0_* ... fam4_* , tg_token
historico	regs2 (registros), total , prox

Limites

O quê	Máximo
Compartimentos/remédios	3
Horários por remédio	6
Familiares	5
Doses no histórico	60

Parte E — Uso autônomo, sem o laptop

Esta parte descreve a montagem do dispenser para funcionamento independente do computador: alimentação pela tomada, com bateria para as faltas de energia. Ela é aplicada depois da validação do protótipo em bancada (E1). Durante o desenvolvimento e os ensaios, utiliza-se a **montagem de bancada** da seção A2: ESP32 alimentado pela USB do computador e servos na fonte de 5 V separada.

E1. Pré-requisitos e validação

A passagem para a montagem autônoma segue as etapas abaixo, na ordem:

Etapa	Procedimento	Critério de aprovação
1. Ensaios funcionais em bancada	Histórico (dose no horário, com atraso, sem acesso e exportação CSV); aviso de 60 s pelo Telegram a cuidador e familiares; abastecimento guiado; reposição por remédio; dois remédios no mesmo horário; migração do cadastro; página no celular	Comportamento igual ao descrito nas Partes C e D, em todos os casos
2. Tempos definitivos	<code>#define MODO_TESTE 0</code> no <code>src/main.cpp</code> (Passo 18), seguido de compilação e gravação	A mensagem <code>ATENCAO: MODO_TESTE</code> ativo não aparece no Serial Monitor
3. Operação assistida	24 horas de funcionamento com os horários reais de medicação, ainda alimentado pelo computador	Nenhuma falha de alarme, abertura de porta ou aviso
4. Endereço fixo na rede	Reserva de DHCP no roteador para o endereço do dispenser	O endereço da página se mantém após o roteador reiniciar
5. Identificação das portas	Etiquetas nas cores da página: 1 azul, 2 verde, 3 roxo	Cores iguais às do cadastro

E2. Por que o laptop não é necessário

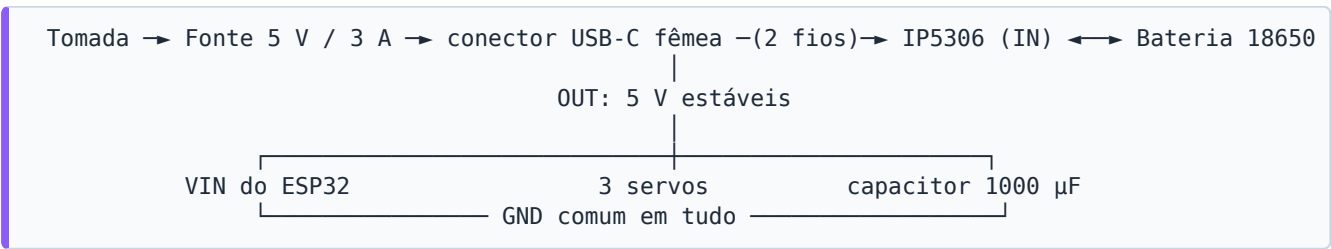
O programa fica **gravado na memória flash** do ESP32 e roda sozinho sempre que a placa recebe energia. Durante os testes, o laptop faz só duas coisas:

1. **Fornece energia** pela porta USB.
2. **Mostra as mensagens** da placa no Serial Monitor (útil para testar, dispensável no uso).

Para funcionar sem o laptop, basta trocar a **fonte de energia**. O código não muda.

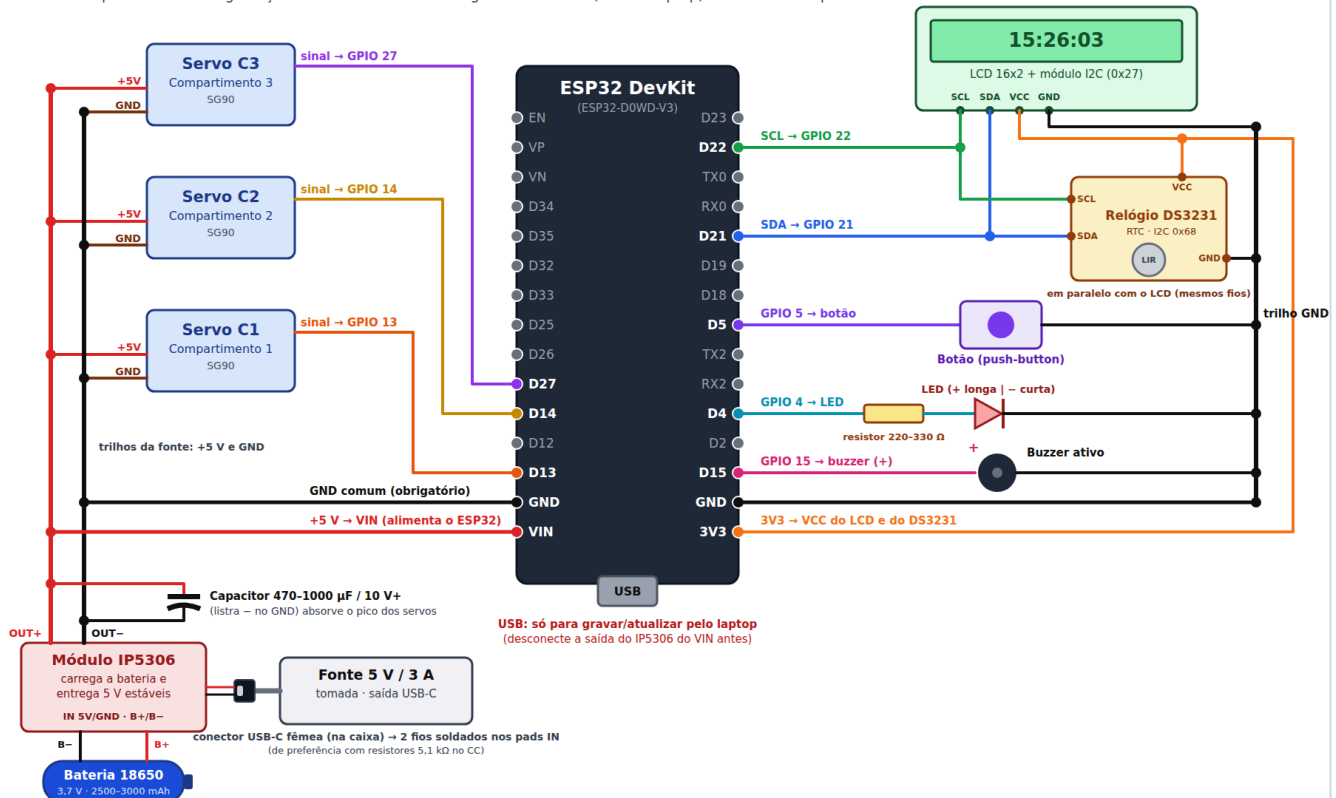
E3. A solução: fonte 5 V/3 A + IP5306 + bateria 18650

Esta é a **única montagem recomendada** para o uso autônomo do Zelo+:



Zelo+ — Montagem autônoma: fonte 5 V/3 A + IP5306 + bateria 18650 + relógio DS3231

Para uso após os testes e a gravação da versão final · a montagem de bancada (USB no laptop) continua válida para testes



Resumo das ligações

Fonte 5 V / 3 A
Bateria 18650
IP5306 OUT+ (5 V)
Capacitor 470-1000 µF
Relógio DS3231
Servos C1 / C2 / C3 — sinal
LCD I2C — SDA / SCL
Botão
LED
Buzzer ativo (+)
IP5306 OUT- (GND)

IP5306: pads IN 5V / GND
IP5306: B+ / B-
VIN + vermelho dos servos
entre +5 V e GND
3V3, GND, GPIO 21, GPIO 22
GPIO 13 / 14 / 27
GPIO 21 / GPIO 22
GPIO 5
GPIO 4 via resistor 220-330 Ω
GPIO 15
GND do ESP32 e dos servos

conector USB-C fêmea, 2 fios
respeitar a polaridade
USB do ESP32 desligado no uso
perto dos servos; respeitar a polaridade
em paralelo com o LCD (I2C, 0x68)
+5V e GND na saída do IP5306
VCC → 3V3, GND → GND
outra perna → GND (sem resistor: INPUT_PULLUP)
perna curta → GND
(-) → GND
GND comum é obrigatório

Legenda

— +5 V (saída do IP5306)
— GND (terra)
— 3,3 V do ESP32
— SDA (dados I2C)
— SCL (clock I2C)
● = ligação (emenda)

Fios que se cruzam SEM ● não estão ligados.
Confira os nomes na serigrafia da sua placa:
a ordem dos pinos varia entre modelos.

Diagrama de ligações da montagem autônoma (fonte 5 V/3 A, IP5306, bateria 18650 e relógio DS3231).
Versão vetorial: <docs/ligacoes-autonomo.svg>.

Como funciona: o IP5306 é um módulo de *power bank* que faz três coisas numa placa só.

Situação	O que o IP5306 faz
Com energia da rua	Passa os 5 V da fonte para o dispenser e, ao mesmo tempo, carrega a bateria
Falta energia	Passa a usar a bateria e eleva de 3,7 V para 5 V — o dispenser continua ligado
Energia volta	Volta a usar a fonte e recarrega a bateria

- A bateria 18650 fica entre 3,0 V (vazia) e 4,2 V (cheia). **Quem entrega os 5 V** ao ESP32 e aos servos é o próprio IP5306 — por isso a tensão da saída não varia com a carga da bateria.
- Use **uma bateria só**, ligada direto em **B+ / B-** do módulo. O IP5306 é feito para **uma célula**: nunca ligue baterias em série (a tensão somaria 6-8,4 V e queimaria o módulo).
- A fonte de **3 A** tem folga: o IP5306 carrega a bateria com até ~2 A e o dispenser consome no pico ~0,6 A.

A entrada de energia: conector USB-C fêmea de painel. O carregador (fonte) é encaixado num **conector USB-C fêmea** preso na parede da caixa do dispenser. Desse conector saem **2 fios** (vermelho e preto) que são **soldados nos pads de entrada (IN) do IP5306**. O ESP32 não usa esse conector: ele recebe os 5 V pela **saída do IP5306 → pino VIN**.

Atenção aos resistores de 5,1 kΩ: fontes USB-C só liberam os 5 V quando o aparelho ligado a elas se identifica, por meio de um **resistor de 5,1 kΩ no pino CC**. Por isso:

- Prefira o conector cujo anúncio diga "**com resistor 5,1 kΩ / 5K1**" ou "**compatível com carregador tipo C (C-C)**".
- **Teste ao receber:** ligue a fonte no conector e meça os 2 fios com o multímetro. Deve marcar **cerca de 5 V**. Se marcar **0 V**, o conector não tem os resistores e a fonte não entrega energia: o dispenser não liga.
- **Plano B:** use um **módulo USB-C fêmea de 6 pinos** (VBUS, GND, CC1, CC2, D+, D-) e solde **um resistor de 5,1 kΩ do CC1 ao GND** e **outro do CC2 ao GND**. Os pinos D+ e D- ficam sem ligação.

E4. Lista de compras

Esta é a lista **só da parte de energia**. A lista **completa do dispenser**, com ilustrações e preços estimados, está na [Parte F](#).

#	Item	Qtd	Especificação / observação
1	Fonte de tomada 5 V / 3 A	1	Saída USB-C , 5 V, 3 A (15 W). Preferir marca conhecida, com certificação Inmetro.
2	Módulo IP5306	1	"Módulo carregador power bank IP5306 5V 2.1A 18650", com pads de solda de entrada (5V/GND), de bateria (B+/B-) e de saída (OUT).
3	Bateria 18650	1	Li-ion 3,7 V, 2.500–3.000 mAh reais (Samsung, LG, Sony/Murata, Panasonic). Desconfiar de "9.800 mAh".
4	Suporte (case) para 1× 18650	1	Com fios vermelho/preto. Evita soldar direto na bateria.
5	Conector USB-C fêmea de painel com 2 fios	1	Preso na caixa; a fonte encaixa nele. Os 2 fios (vermelho/preto) são soldados nos pads de entrada do IP5306. De preferência com resistores 5,1 kΩ no CC (ver acima).
6	Capacitor eletrolítico 1000 µF / 16 V	1	Entre +5 V e GND, perto dos servos. Tem polaridade (faixa "–" no GND).
7	Fio 22 AWG vermelho e preto	~1 m cada	Saída do IP5306 → VIN do ESP32 e servos.
8	Placa perfurada ou barramento de bornes	1	Para fazer os "trilhos" de +5 V e GND com solda (mais firme que protoboard para uso contínuo).
9	Termo-retrátil sortido	1 kit	Isolar as emendas soldadas.
10	Multímetro	1	Conferir polaridade e os 5 V da saída antes de ligar o ESP32.
11	Ferro de solda + estanho	—	Para os pads do IP5306 e as emendas.
12	<i>Módulo relógio DS3231 + bateria LIR2032 (opcional)</i>	1	Ver E9 : mantém a hora certa sem internet.

Módulos IP5306 costumam vir também com uma **saída USB-A**. Ela entrega os mesmos 5 V, mas para o dispenser use os **pads de saída soldados**: a ligação fica firme e não depende de um conector que pode se soltar.

E5. Montagem passo a passo

Monte **sem a bateria e sem a fonte ligadas**. Só energize nas etapas indicadas.

1. **Conector de entrada:** fixe o conector USB-C fêmea na caixa e solde os 2 fios nos pads de **entrada** do IP5306: **vermelho → 5V (IN+)** e **preto → GND (IN-)**. Isole com termo-retrátil. Antes, faça o teste dos 5 V no conector (ver E3).
2. **Bateria:** solde os fios do suporte da 18650 nos pads **B+** (vermelho) e **B-** (preto). Confira a polaridade com o multímetro antes de encaixar a bateria.
3. **Saída:** solde um fio vermelho em **OUT+ (5V)** e um preto em **OUT- (GND)** e leve até a placa perfurada, formando os **trilhos de +5 V e GND**.
4. **Teste sem o ESP32:** encaixe a bateria e ligue a fonte na tomada. Com o multímetro, meça os trilhos: devem marcar **entre 4,9 V e 5,2 V**. Desligue tudo.
5. **Capacitor:** solde o capacitor de 1000 µF entre os trilhos, **perto dos servos** (faixa "-" no GND).
6. **Servos:** fio **vermelho** de cada servo no trilho de +5 V, **marrom** no GND. Os fios de sinal continuam nos GPIO 13, 14 e 27.
7. **ESP32:** trilho de +5 V → pino **VIN**; trilho de GND → pino **GND** do ESP32. **Não ligue o cabo USB do ESP32** — a energia entra só pelo VIN.
8. **Resto da montagem** (LCD, botão, LED, buzzer): igual à bancada.
9. **Ligar:** fonte na tomada. O LCD deve mostrar "Iniciando...", depois o IP, depois o relógio.

E6. Teste de falta de energia

Com o dispenser funcionando (relógio no LCD):

1. **Tire a fonte da tomada.** O dispenser deve **continuar ligado**, agora pela bateria.
2. Observe o LCD:
3. Se o relógio **continua normal**, a troca foi perfeita.
4. Se aparecer **"Iniciando..."**, o ESP32 reiniciou no instante da troca. Nada se perde (cadastro, horários e histórico ficam salvos) e ele volta sozinho em poucos segundos. Para evitar, confira se o capacitor está perto dos servos e firme.
5. **Recoloque a fonte.** O dispenser segue ligado e a bateria volta a carregar (os LEDs do módulo indicam a carga).
6. Faça um **alarme de teste** com o dispenser na bateria (servo abrindo e fechando), para confirmar que a bateria aguenta o pico dos servos.

Outros cuidados com o IP5306:

- No modo bateria, ele **se desliga sozinho se o consumo ficar abaixo de ~50 mA**. O dispenser com Wi-Fi consome mais que isso, então não deve acontecer — mas se a bateria **zerar**, algumas placas só religam a saída ao **apertar o botãozinho** do módulo ou quando a energia da rua volta.
- Carregando e alimentando ao mesmo tempo, o módulo **esquenta um pouco**: deixe-o com ventilação e sem encostar na bateria.

E7. Autonomia da bateria

Conta para **uma 18650 real de 2.500–3.000 mAh**:

Etapa	Valor
Energia da bateria	2.500 mAh × 3,7 V ≈ 9,3 Wh
Perda na elevação para 5 V (~85%)	sobram ≈ 7,9 Wh
Consumo médio do dispenser (ESP32 com Wi-Fi, LCD, servos parados)	≈ 0,75 a 1 W
Autonomia sem energia da rua	≈ 8 a 10 horas

Os servos se mexem só alguns segundos por dia e quase não pesam na conta. Para saber o número real da sua bateria: carregue até o fim, tire da tomada e anote quanto tempo o dispenser fica ligado.

E8. Falta de energia e de internet

Situação	O que acontece
Falta energia (até ~8-10 h)	O dispenser continua funcionando pela bateria . Os alarmes tocam; o Telegram depende de a internet (roteador) também estar ligada.
Falta energia por mais tempo (bateria acaba)	O dispenser desliga. Cadastro, horários e histórico ficam salvos ; horários que passarem desligado não tocam depois.
Energia volta, com internet	Liga sozinho, conecta no Wi-Fi, acerta a hora e volta ao normal; a bateria recarrega.
Energia volta, sem internet — sem o DS3231	Sem hora certa, os alarmes não tocam até a internet voltar (o dispenser fica no modo de configuração).
Energia volta, sem internet — com o DS3231	Liga com a hora do relógio (Sem internet / Hora do relógio no LCD): os alarmes tocam normalmente . O Wi-Fi é tentado a cada 30 s; o Telegram volta junto com a internet.
Internet cai, energia ok	O relógio interno continua contando: os alarmes tocam normalmente . Só as mensagens do Telegram não saem.

E9. Módulo de relógio DS3231

Já suportado pelo programa (Passo 19). O módulo é **opcional**: sem ele, tudo funciona pela internet. Com ele, o dispenser **sabe a hora mesmo sem internet**, inclusive depois de a bateria acabar e a energia voltar.

O que é: uma plaquinha com um relógio de alta precisão e uma **bateria tipo moeda**. Ela continua contando o tempo por anos, mesmo com o dispenser desligado.

Característica	Valor
Precisão	±2 ppm (cerca de 1 minuto por ano)
Comunicação	I2C, endereço 0x68 (não conflita com o LCD, que usa 0x27)
Alimentação	3,3 V (pino 3V3 do ESP32)
Bateria	LIR2032 (recarregável) — ver aviso abaixo
Preço aproximado	R\$ 26 a R\$ 31 (ver F4)

Ligação: no **mesmo barramento I2C do LCD**, em paralelo: os fios SDA e SCL do DS3231 se juntam aos do LCD (ponto ● no diagrama da seção E3), e o VCC sai do mesmo fio de 3V3.

DS3231	ESP32
VCC	3V3
GND	GND
SDA	GPIO 21 (junto com o SDA do LCD)
SCL	GPIO 22 (junto com o SCL do LCD)

Muitos módulos DS3231 vêm com um circuito que tenta **recarregar** a bateria. Com uma **CR2032 comum (não recarregável)**, isso pode estragar a bateria. Use uma **LIR2032** (recarregável) ou peça para alguém retirar o resistor/diodo de carga do módulo.

Como o programa usa o módulo (código completo no [Passo 19](#)):

1. **Ao ligar:** procura o DS3231. Se ele tiver uma hora válida, acerta o relógio do ESP32 na hora, sem esperar a internet. Os alarmes passam a funcionar imediatamente.
2. **Com internet:** a cada acerto de hora pelo NTP (ao conectar e depois a cada ~1 h), **grava a hora certa no DS3231**, para ele nunca se afastar da hora certa.
3. **Sem internet:** segue pelo DS3231. Os alarmes funcionam normalmente; o Wi-Fi é tentado a cada 30 s e só o Telegram espera a internet voltar.
4. **Sem o módulo:** o programa percebe na hora de ligar e segue como antes.

Instalar o módulo:

1. **Desligue o dispenser da energia** (fonte e bateria/IP5306).
2. Coloque a **LIR2032** no módulo e ligue os 4 fios: VCC → 3V3, GND → GND, SDA → GPIO 21, SCL → GPIO 22.
3. Religue **com internet**. No Serial Monitor deve aparecer `DS3231 sem hora valida - aguardando a internet` e, em seguida, `DS3231: hora gravada (vinda da internet)`. Na página, o rodapé mostra **"Relógio DS3231: conectado"**.

Mensagens no Serial Monitor

Mensagem	Significado
DS3231 nao encontrado - usando so a internet	Módulo ausente ou mal ligado (confira SDA/SCL).
DS3231 sem hora valida - aguardando a internet	Módulo novo ou bateria LIR2032 descarregada.
DS3231: hora carregada do relógio	Hora certa já ao ligar.
DS3231: hora gravada (vinda da internet)	O NTP acertou a hora e o módulo foi atualizado.
Sem internet: alarmes pela hora do DS3231	Ligou sem internet, funcionando pelo relógio.

Ensaios do relógio:

Ensaio	Resultado esperado
Página aberta com o módulo instalado	Rodapé mostra "Relógio DS3231: conectado"
Religamento sem internet (roteador desligado)	LCD mostra <code>Sem internet / Hora do relógio</code> e, em seguida, a hora correta
Horário de dose com o roteador desligado	O alarme toca no horário
Retorno da internet	Em até ~30 s o dispenser reconecta; a página e o Telegram voltam a funcionar
Religamento sem o módulo	Rodapé mostra "não instalado"; hora obtida pela internet

E10. Atualizar o programa depois de montado

A montagem autônoma não impede atualizações:

1. **Desconecte a saída do IP5306 do VIN** do ESP32 (solte o fio do VIN ou use um borne que se possa abrir). **Nunca ligue o USB e o VIN ao mesmo tempo.**
 2. Ligue o **USB do ESP32 no laptop** e grave a versão nova (Build → Upload), como na bancada.
 3. Tire o USB, **religue o fio do VIN** e o dispenser volta a funcionar pela fonte e pela bateria.
-

Parte F — Lista completa de componentes, com preços

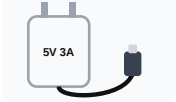
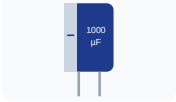
Relação dos componentes necessários para montar o Zelo+ na **versão autônoma** (tomada + bateria, Parte E). As ilustrações são desenhos simplificados para identificar cada peça.

Sobre os preços: valores pesquisados em lojas brasileiras de eletrônica em **setembro de 2026** (ver [Referências](#)). Itens marcados com * não tiveram o preço exibido em loja nacional na pesquisa: o valor é uma **estimativa** a partir de preços internacionais convertidos. Preços mudam com frequência e **não incluem frete**. Ferramentas **não** entram na conta (ver [F5](#)).

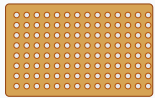
F1. Eletrônica do dispenser

Componente e códigos	Ilustração	Função	Preço de referência
Placa ESP32 DevKit V1 — 30 pinos · 1 un. ESP32-WROOM-32 · chip ESP32-D0WD-V3 · USB CP2102		O "cérebro": roda o programa, cria a página web, conecta no Wi-Fi e manda os avisos.	R\$ 54,90
Micro servo SG90 9 g · 3 un. TowerPro SG90 · 180° · 1,6 kgf·cm · 4,8–6 V		Abre e fecha a porta de cada compartimento (um servo por compartimento).	R\$ 13,33–16,90 cada R\$ 40–51 (3 un.)
Display LCD 16x2 com módulo I2C · 1 un. HD44780 + PCF8574 · endereço 0x27 · fundo azul		Mostra o relógio, o nome do remédio e os avisos (abrindo, fechando, dose pendente).	R\$ 24,90
Buzzer ativo 5 V · 1 un. 12 mm · "auto-oscilante" · 3,5–5 V		Apita no horário do remédio (alarme sonoro).	R\$ 2,50
LED difuso 5 mm vermelho · 1 un.		Pisca junto com o buzzer (alarme visual).	R\$ 0,15
Resistor 220 Ω, ¼ W · pacote com 10 faixas: vermelho-vermelho-marrom-dourado		Limita a corrente do LED para ele não queimar (usa 1; sobram reservas).	R\$ 0,90 (10 un.)
Chave tátil (push button) 12x12 mm com capa · kit com 10		O botão que o paciente aperta para abrir e fechar os compartimentos (usa 1).	R\$ 10,20 (10 un.)

F2. Energia autônoma

Componente e códigos	Ilustração	Função	Preço de referência
Fonte de tomada 5 V / 3 A, saída USB-C · 1 un. bivolt · 15 W · com certificação Inmetro		Energia da rua para o dispenser e para carregar a bateria.	R\$ 35,98-50
Módulo IP5306 5 V 2,1 A (power bank) · 1 un. IP5306 · entrada USB-C · pads IN, B+/B-, OUT		Carrega a bateria e entrega 5 V estáveis, com ou sem energia da rua.	R\$ 15-25 *
Bateria Li-ion 18650 · 1 un. Onistek ON-18650 · 3,7 V · rótulo 3.800 mAh (real provável 1.500-2.500 mAh)		Mantém o dispenser ligado se faltar energia (~4-10 h, conforme a capacidade real).	R\$ 17,15
Suporte para 1 bateria 18650 · 1 un. com fios vermelho/preto		Segura a bateria e evita soldar direto nela.	R\$ 9,17
Conector USB-C fêmea de painel, com 2 fios · 1 un. "jack USB tipo C fêmea com rabicho 2 fios" · 5 V · $\geq 2,5$ A · de preferência com resistores 5,1 k Ω		Entrada de energia na caixa: a fonte encaixa nele e os 2 fios são soldados na entrada (IN) do IP5306.	R\$ 12-30
Resistor 5,1 kΩ, 1/4 W · pacote com 10 (<i>plano B: só se usar o módulo USB-C de 6 pinos</i>) faixas: verde-marrom-vermelho-dourado		Um em cada pino CC (CC1 e CC2) até o GND: faz a fonte USB-C liberar os 5 V.	R\$ 0,60 (10 un.)
Capacitor eletrolítico 1000 μF / 16 V · 1 un. 105 °C · tem polaridade (faixa "-")		Absorve o pico de corrente dos servos e evita que o ESP32 reinicie.	R\$ 0,73-3

F3. Montagem e ligações

Componente e códigos	Ilustração	Função	Preço de referência
Placa perfurada ilhada 7×9 cm · 1 un. fenolite ou fibra de vidro		Base firme, soldada, para os trilhos de +5 V e GND (mais durável que protoboard).	R\$ 3,90–12,50
Jumpers macho/fêmea 20 cm · 1 kit (40 un.)		Ligações entre o ESP32, o LCD, os servos e a placa.	R\$ 8,90
Fio flexível 22 AWG · vermelho e preto, ~2 m de cada		Ligações de energia: saída do IP5306 → VIN e servos.	R\$ 1,20–3 por metro R\$ 5–12 (4 m)
Tubo termo-retrátil · 1 kit sortido		Isola as emendas soldadas (evita curto-circuito).	R\$ 16,88–43
Cabo USB-A → micro-USB (dados) · 1 un.		Gravar e atualizar o programa pelo laptop.	a partir de R\$ 11,69

F4. Relógio sem internet (opcional)

Componente e códigos	Ilustração	Função	Preço de referência
Módulo RTC DS3231 · 1 un. DS3231 + EEPROM AT24C32 · I2C 0x68 · 3,3–5 V		Guarda a hora certa mesmo sem internet e sem energia (ver E9).	R\$ 26–30,91
Bateria LIR2032 (recarregável) · 1 un. 3,6 V · tipo moeda		Alimenta o relógio do DS3231 quando o dispenser está desligado.	R\$ 11,90–28,50

F5. Ferramentas necessárias (fora do custo)

As ferramentas **não entram na soma** do projeto: são de uso geral e costumam ser emprestadas ou já existir em casa ou no laboratório da escola. Mesmo assim, são **necessárias** para a montagem autônoma, que tem soldas em peças pequenas.

Ferramenta	Ilustração	Para quê	Modelo recomendado e ajustes
Ferro de solda com ajuste de temperatura		Soldar os fios nos pads do IP5306, no conector USB-C e na placa perfurada.	Estação ou ferro com controle de temperatura , 50–60 W, faixa ~200–480 °C (ex.: Hikari HK-936B ou Yaxun 936). • Temperatura: 320–350 °C para os pads pequenos (IP5306, pinos, resistores); até 370 °C para os fios 22 AWG e os trilhos da placa. • Ponta: cônica fina (0,5–1 mm) ou chanfrada pequena (~1,6 mm). • Evite ferros simples sem ajuste (ex.: linhas "60 W" comuns, que passam de 500 °C): o excesso de calor pode soltar os pads do IP5306 e danificar a bateria se encostar nela.
Estanho (solda) 60/40 com fluxo, 0,5 mm		Material da solda.	Fio 0,5 mm (fino, próprio para peças pequenas), liga 60/40 com fluxo resinoso (ex.: Cobix 0,5 mm RA T2 , carretel pequeno de 250 g ou tubinho).
Multímetro digital		Conferir os 5 V da saída e a polaridade antes de ligar o ESP32.	Modelo básico com tensão DC e teste de continuidade (bip) (ex.: Hikari HM-1001 ou Minipa ET-1002).
Acessórios de bancada	—	Preparar fios e corrigir erros.	Suporte com esponja para o ferro (costuma vir com a estação) · alicate de corte rente pequeno · decapador para fio 22 AWG · malha dessoldadora ou sugador · pinça · "terceira mão" com lupa (opcional) · isqueiro ou soprador para o termo-retrátil.
Segurança	—	—	Óculos de proteção , ambiente ventilado (fumaça do fluxo) e lavar as mãos depois de usar estanho com chumbo. Nunca soldar com a bateria encaixada no suporte.

F6. Resumo do investimento

Grupo	Faixa de preço
F1. Eletrônica do dispenser	R\$ 134 – 147
F2. Energia autônoma (<i>com a bateria</i>)	R\$ 90 – 135
F3. Montagem e ligações	R\$ 46 – 96
Total para montar o dispenser	≈ R\$ 270 – 378
F4. Relógio sem internet (opcional, à parte)	+ R\$ 38 – 60

Não inclui: ferramentas (F5), frete e a estrutura física do dispenser (caixa, portas e divisórias dos compartimentos), que depende do material escolhido (MDF, acrílico ou impressão 3D). O custo da caixa em impressão 3D e em plástico injetado, e o custo por unidade em produção, estão nas Partes **B** (desenho técnico) e **G** (custos).

Observações sobre os componentes:

- **ESP32, bateria e fonte** são os componentes com maior incidência de imitações no mercado; recomenda-se adquiri-los de fornecedores especializados em eletrônica.
 - Uma bateria 18650 genuína tem no máximo cerca de 3.500 mAh; valores como "9.800 mAh" ou "12.000 mAh" no rótulo indicam produto falsificado.
 - Resistores, botões e termo-retrátil são vendidos em pacotes; o preço indicado é o do pacote.
-

Parte G — Custo de produção (estimativa para viabilidade)

Natureza desta parte: estimativa para avaliar a **viabilidade** de produzir o Zelo+ em série. **Data-base: setembro de 2026.** Não é orçamento: os valores são faixas (mínimo-máximo) calculadas a partir de premissas declaradas, e cada número pode ser feito trocando a premissa correspondente. As medidas do gabinete são as do desenho técnico (Parte B). As fontes dos valores estão na seção [Referências](#).



Imagem de referência — conceito visual do produto final (ilustrativo). Não é o protótipo atual.

G1. Premissas e método

Custo por unidade = componentes + gabinete + ferragens + montagem + embalagem + **investimentos amortizados** (investimento ÷ quantidade produzida).

Premissa	Valor adotado	Origem
Data-base	setembro de 2026	—
Câmbio	US\$ 1 = R\$ 5,16	cotação de 24/09/2026 (faixa de 52 semanas: R\$ 4,88-5,61)
Fator de nacionalização (componentes importados)	1,8 × o preço de fábrica em dólar	frete e seguro (~8-10 %) + imposto de importação (0-16 %) + IPI + PIS/COFINS + ICMS; simplificação de ordem de grandeza
Custo de US\$ 1 de componente posto no Brasil	$5,16 \times 1,8 = \text{R\$ } 9,29$	cálculo
Energia elétrica	R\$ 0,849/kWh	tarifa residencial média projetada pela Aneel para 2026
Mão de obra de montagem	R\$ 20/h	salário de montador + encargos (~70 %) ÷ 176 h/mês
Volumes analisados	100, 1.000 e 10.000 unidades	lote piloto, pequena série, série
Arredondamento	valores finais arredondados ao real	—

G2. Volume de plástico do gabinete

Com as medidas do desenho técnico (Parte B), o volume de plástico é a soma de área × espessura de cada parte:

Parte	Conta (mm)	Volume
Paredes externas	perímetro 680 × altura 100 × 3,0	204,0 cm ³
Fundo	210 × 130 × 2,5	68,2 cm ³
Piso dos compartimentos	204 × 75,7 × 2,5	38,6 cm ³
Divisórias entre compartimentos	2 × 75,7 × 50,5 × 2,0 + apoio das tampas (2 × 75,7 × 5 × 1,45)	16,4 cm ³
Divisória da área técnica	204 × 50,5 × 2,0	20,6 cm ³
3 tampas móveis	3 × 65 × 75 × 2,0	29,2 cm ³
Tampa da área técnica	203,3 × 45,6 × 2,0	18,5 cm ³
Detalhes (dobradiças, fixações, moldura do LCD, logotipo)	+ 12 %	47,5 cm ³
Total		≈ 443 cm³

Material	Densidade	Massa da peça pronta
PLA (impressão 3D)	1,24 g/cm ³	443 × 1,24 = ≈ 549 g (+ 8 % de perdas = 593 g de filamento)
ABS (injeção)	1,05 g/cm ³	443 × 1,05 = ≈ 465 g

G3. Gabinete em impressão 3D

Equipamento de referência: impressora de mesa **300 × 300 mm** (ex.: Creality K1 Max, ~R\$ 6.776). O gabinete de 210 × 130 mm cabe inteiro na mesa.

Item	Conta (cenário médio)	Valor
Tempo de impressão	549 g ÷ 30 g/h	18,3 h
Filamento PLA	0,593 kg × R\$ 110/kg	R\$ 65,28
Energia	18,3 h × 0,20 kW × R\$ 0,849	R\$ 3,11
Depreciação da impressora	18,3 h × (R\$ 6.776 ÷ 5.000 h de vida útil)	R\$ 24,82
Manutenção (bicos, mesa, correias)	18,3 h × R\$ 0,30	R\$ 5,49
Reserva para falhas de impressão	10 % dos itens acima	R\$ 9,87
Acabamento (retirar suportes, lixar)	0,5 h × R\$ 20	R\$ 10,00
Total por gabinete		R\$ 118,57

Faixa: de **R\$ 100** (filamento a R\$ 90/kg, 35 g/h, 15,7 h) até **R\$ 157** (R\$ 140/kg, 20 g/h, 27,5 h).

- **Conferência:** sem o acabamento, o custo dá **R\$ 0,20 por grama**, dentro da faixa de custo real de produção citada pelo mercado para 2026 (R\$ 0,18–0,30/g).
- **Capacidade:** a ~18,3 h por gabinete, uma impressora produz **~380 gabinetes/ano** (80 % do tempo em uso). Para 1.000 unidades/ano são necessárias ~3 impressoras.
- **Quando usar:** lote piloto e pequena série. Não há investimento em molde, e o desenho pode mudar a qualquer momento.

G4. Gabinete em plástico injetado

Moldes necessários (aço P20, que aguenta a produção em série; valores de mercado para o Brasil em 2026):

Molde	Peça	Investimento estimado
1	Corpo azul (compartimentos, divisórias, área técnica)	R\$ 75-130 mil
2	Base preta (recortes do LCD, botão, acrílico e USB-C)	R\$ 50-90 mil
3	Molde "família": 3 tampas móveis + tampa técnica	R\$ 40-75 mil
—	Projeto do molde (DFM) e ajustes de teste	R\$ 15-30 mil
Total em aço		R\$ 180-325 mil
Alternativa em alumínio (~5-10 mil injeções; ~50 % do custo do aço, mais o DFM)		R\$ 98-178 mil

A referência de mercado é de R\$ 25-70 mil para o molde de uma peça pequena e simples, de uma cavidade. O corpo e a base do Zelo+ são peças médias, com divisórias, recortes e nervuras, por isso as faixas acima são maiores.

Custo de injetar um gabinete (sem o molde):

Item	Conta	Mínimo	Máximo
Resina ABS	0,465 kg × 1,05 (canal de injeção) × R\$ 13-18/kg	R\$ 6,35	R\$ 8,79
Pigmento (masterbatch azul e preto, 2 %)	9,8 g × R\$ 35/kg	R\$ 0,34	R\$ 0,34
Hora-máquina (injetora + operador + energia)	ciclos de 50 + 45 + 35 s = 130 s × R\$ 150-250/h × 1,05 (refugo)	R\$ 5,69	R\$ 9,48
Logotipo "Zelo+" (tampografia)	por peça	R\$ 1,00	R\$ 2,00
Inspeção	por conjunto	R\$ 0,50	R\$ 0,50
Total por gabinete		R\$ 13,88	R\$ 21,11

Com o molde amortizado (investimento ÷ quantidade + custo de injeção):

Quantidade	Molde de alumínio	Molde de aço
500	R\$ 209-376	R\$ 374-671
1.000	R\$ 111-199	R\$ 194-346
2.000	R\$ 63-110	R\$ 104-184
5.000	R\$ 33-57	R\$ 50-86
10.000	R\$ 24-39	R\$ 32-54

Ponto de equilíbrio (impressão 3D × injeção):

quantidade = investimento no molde ÷ (custo 3D - custo injetado)

diferença por unidade: de R\$ 100 - 21 = R\$ 79 até R\$ 157 - 14 = R\$ 143

molde de alumínio: R\$ 98-178 mil ÷ R\$ 79-143 ≈ 680 a 2.240 unidades

molde de aço: R\$ 180-325 mil ÷ R\$ 79-143 ≈ 1.260 a 4.100 unidades

Conclusão: Abaixo de ~1.000 unidades, a impressão 3D é mais barata. Acima de ~4.000, o molde de aço compensa com folga.

G5. Componentes eletrônicos no atacado

Preços de fábrica em dólar para lotes de ~500–1.000 unidades, multiplicados por R\$ 9,29 (G1). A fonte é comprada no Brasil, porque precisa da certificação do Inmetro.

Cenário 1 — os mesmos módulos do protótipo

Componente	US\$ (fábrica)	R\$ posto no Brasil
ESP32 DevKit V1	2,50–3,50	23,22–32,51
3 servos SG90	2,10–3,00	19,50–27,86
LCD 16x2 + I2C	1,50–2,20	13,93–20,43
Buzzer, LED, resistor e botão	0,20–0,35	1,86–3,25
Módulo IP5306	0,60–1,00	5,57–9,29
Bateria 18650	1,50–2,50	13,93–23,22
Suporte 18650	0,10–0,20	0,93–1,86
Conector USB-C de painel	0,25–0,50	2,32–4,64
Capacitor 1000 µF	0,05	0,46
Placa, fios e termo-retrátil	0,80–1,50	7,43–13,93
Relógio DS3231 + LIR2032	0,90–1,50	8,36–13,93
Fonte 5 V/3 A USB-C (Inmetro, compra nacional)	—	15,00–22,00
Total		R\$ 112,52–173,39

Cenário 2 — placa de circuito própria (ESP32, carregador, USB-C, relógio, buzzer, botão e LED numa única placa, montada pela fábrica)

Componente	R\$ posto no Brasil
Módulo ESP32-WROOM-32E	18,58–24,15
Placa + montagem automática (IP5306, USB-C, relógio, buzzer, botão, LED)	27,86–51,08
LCD 16x2 + I2C	12,07–17,65
3 servos SG90	19,50–27,86
Bateria 18650 + suporte	14,86–25,08
Cabos internos	2,79–5,57
Fonte 5 V/3 A USB-C (Inmetro)	15,00–22,00
Total	R\$ 110,67–173,39

As peças custam praticamente o mesmo nos dois cenários. O ganho da placa própria está na **montagem** (de ~55 para ~20 min por unidade), na **confiabilidade** (sem fios soltos) e no **tamanho**.

Protótipo (Parte F, varejo) × produção (atacado): **R\$ 278–378 → R\$ 111–173**, cerca de **55–60 % menos**.

G6. Custo por unidade em três escalas

Item	Lote piloto — 100 un.	Pequena série — 1.000 un.	Série — 10.000 un.
Arranjo	módulos + impressão 3D	placa própria + impressão 3D	placa própria + injeção (aço)
Componentes	R\$ 135-208 ¹	R\$ 111-173	R\$ 111-173
Gabinete	R\$ 100-157	R\$ 100-157	R\$ 14-21
Ferragens (parafusos, pinos das portas, pés, visor do LCD)	R\$ 5-9	R\$ 5-9	R\$ 5-9
Montagem e teste	55 min → R\$ 18	20 min → R\$ 7	20 min → R\$ 7
Embalagem e manual	R\$ 8-14	R\$ 6-12	R\$ 6-12
Projeto da placa (R\$ 5-20 mil) amortizado	—	R\$ 5-20	R\$ 0,50-2
Moldes de aço (R\$ 180-325 mil) amortizados	—	—	R\$ 18-33
Custo de fabricação	R\$ 267-406	R\$ 234-378	R\$ 161-257
Homologação Anatel (R\$ 10-30 mil) amortizada	R\$ 100-300	R\$ 10-30	R\$ 1-3
Custo total por unidade	R\$ 367-706	R\$ 244-408	R\$ 162-260

¹ Componentes do cenário 1 com acréscimo de 20 % por comprar lotes pequenos.

Leitura dos resultados:

- Na série, o **gabinete** cai de R\$ 100-157 (impressão 3D) para R\$ 14-21 (injeção), e a eletrônica passa a ser o principal custo.
- No lote piloto, a **homologação** pesa mais que qualquer peça. Por isso ela só faz sentido quando já existe plano de produzir em quantidade.
- Entre 1.000 e ~4.000 unidades, a escolha entre impressão 3D, molde de alumínio e molde de aço depende da quantidade prevista (ponto de equilíbrio na G4).

G7. Investimentos iniciais (uma vez só)

Investimento	Valor estimado	Necessário a partir de
Projeto e protótipos da placa de circuito	R\$ 5-20 mil	pequena série
Homologação Anatel (produto com Wi-Fi)	R\$ 10-30 mil	qualquer venda
Impressoras 3D para ~1.000 un./ano (~3 × R\$ 6.776)	≈ R\$ 20 mil	pequena série (se produção própria)
Moldes de injeção (aço)	R\$ 180-325 mil	série
Consulta de enquadramento na Anvisa	variável	antes da venda

G8. Preço de venda e viabilidade

Produtos de hardware costumam chegar ao consumidor por **2,5 a 4 vezes o custo de fabricação**. Essa margem cobre impostos sobre a venda, distribuição, garantia, assistência técnica e lucro.

Escala	Custo total por unidade	Preço de venda estimado (× 2,5 a × 4)
Lote piloto (100)	R\$ 367-706	R\$ 920-2.820
Pequena série (1.000)	R\$ 244-408	R\$ 610-1.630
Série (10.000)	R\$ 162-260	R\$ 400-1.040

Síntese: em série, o Zelo+ ficaria na faixa de **R\$ 400 a R\$ 1.040** para o consumidor. O **custo de entrada** (moldes, placa própria e homologação) fica entre **R\$ 195 mil e R\$ 375 mil**, e só se paga com volumes a partir de alguns milhares de unidades. Antes disso, o caminho viável é o **lote piloto em impressão 3D**, que exige pouco investimento.

G9. Limitações da estimativa

- **Dimensões:** as medidas são as do desenho técnico (Parte B). Mudanças no desenho (espessuras, nervuras, rampa no fundo) alteram o volume de plástico e os custos do gabinete.
- **Preços:** são faixas de mercado, não cotações. Os moldes, em especial, exigem orçamento com o desenho 3D final.
- **Câmbio:** variou de R\$ 4,88 a R\$ 5,61 em 52 semanas ($\pm 9\%$), o que afeta direto o custo dos componentes.
- **Impostos:** o fator de 1,8 é uma simplificação; o valor real depende do regime tributário da empresa e da classificação fiscal de cada peça.
- **Durabilidade:** para uso diário por anos, recomenda-se trocar o SG90 pelo **MG90S** (engrenagem de metal): **+R\$ 3-5 por servo**.
- **Não incluídos:** frete até o cliente, marketing, impostos sobre a venda e custos administrativos. Bateria de lítio tem regras próprias de transporte.
- **Anvisa:** é preciso confirmar se um dispenser com alarme de medicação se enquadra como dispositivo médico. Se sim, custos e prazos aumentam.

Referências

Fontes consultadas para os preços, as especificações dos componentes e os parâmetros de custo utilizados nas Partes F e G (consulta em setembro de 2026).

AMAZON.COM.BR. **Display LCD 16x2 com módulo I2C PCF8574**. Disponível em:

<https://www.amazon.com.br/Display-PCF8574-Endere%C3%A7o-Controlador-80x35mm/dp/B0H346LHKM>. Acesso em: 27 set. 2026.

AMAZON.COM.BR. **Placa breakout USB tipo C fêmea, 6 pinos**. Disponível em:

<https://www.amazon.com.br/naughtystarts-pe%C3%A7as-breakout-conector-direito/dp/B0B19TP2MX>. Acesso em: 27 set. 2026.

BAÚ DA ELETRÔNICA. **Rolo de solda estanho 500 g, 0,5 mm, Cobix**. Disponível em:

<https://www.baudaeletronica.com.br/produto/rolo-de-solda-estanho-500g-05mm-cobix.html>. Acesso em: 27 set. 2026.

CASA DA ROBÓTICA. **Ferro de solda Hikari SC-60, 60 W**. Disponível em:

<https://www.casadarobotica.com/prototipagem-e-ferramentas/prototipagem/soldas/ferro-de-solda-hk-plus-hikari-profissional-sc-60w-220v>. Acesso em: 27 set. 2026.

CHEN HSONG BRASIL. **Custo da hora-homem-máquina para operadores de injetoras**. Disponível em: <https://chenhsong.com.br/news/custo-da-hora-homem-maquina-para-operadores-de-injetoras/>.

Acesso em: 27 set. 2026.

CURTO CIRCUITO. **Servo motor 9 g SG90**. Disponível em: <https://curtocircuito.com.br/servo-motor-9g-sg90.html>. Acesso em: 27 set. 2026.

EBAY. **Módulo carregador IP5306, 2,1 A**. Disponível em: <https://www.ebay.de/itm/134628665112>.

Acesso em: 27 set. 2026.

ELETROGATE. **Jumpers macho-fêmea, 40 unidades de 20 cm**. Disponível em:

<https://www.eletrogate.com/jumpers-macho-femea-40-unidades-de-20-cm>. Acesso em: 27 set. 2026.

ELETROGATE. **LED difuso 5 mm vermelho**. Disponível em: <https://www.eletrogate.com/led-difuso-5mm-vermelho>. Acesso em: 27 set. 2026.

ELETROGATE. **Módulo WiFi ESP32 Bluetooth, 30 pinos**. Disponível em:

<https://www.eletrogate.com/modulo-wifi-esp32-bluetooth-30-pinos>. Acesso em: 27 set. 2026.

ELETROGATE. **Módulo serial I2C para display LCD**. Disponível em:

<https://www.eletrogate.com/modulo-serial-i2c-para-display-lcd-para-arduino>. Acesso em: 27 set. 2026.

ELETROGATE. **Resistor 220 Ω , 1/4 W, 10 unidades**. Disponível em:

<https://www.eletrogate.com/resistor-220r-1-4w-10-unidades>. Acesso em: 27 set. 2026.

ELETROGATE. **Resistor 5,1 k Ω , 1/4 W, 10 unidades**. Disponível em:

<https://www.eletrogate.com/resistor-5k1-1-4w-10-unidades>. Acesso em: 27 set. 2026.

FERMARC. **Placa de circuito perfurada face simples, 7 x 9 cm**. Disponível em:

<https://www.fermarc.com/placa-de-circuito-perfurada-face-simples-7x9>. Acesso em: 27 set. 2026.

FERRO DE SOLDA PROFISSIONAL. **Ferros de solda com controle de temperatura**. Disponível em:

<https://ferrodesoldaprofissional.com.br/ferro-de-solda/>. Acesso em: 27 set. 2026.

GALPÃO DAS MÁQUINAS. **Como calcular o custo por peça na impressão 3D**. Disponível em:

<https://galpaodasmaquinas.com.br/blog/plastico/custo-pecas-impressora-3d/>. Acesso em: 27 set. 2026.

INFOMONEY. **Aneel projeta alta média de 8% para tarifas de consumidores de energia**

elétrica. Disponível em: <https://www.infomoney.com.br/economia/aneel-projeta-alta-media-de-8-para-tarifas-de-consumidores-de-energia-eletrica/>. Acesso em: 27 set. 2026.

INFOMONEY. **O que esperar do dólar em setembro? Veja até onde a moeda pode ir**. Disponível

em: <https://www.infomoney.com.br/mercados/o-que-esperar-do-dolar-em-setembro-veja-ate-onde-a-moeda-pode-ir/>. Acesso em: 27 set. 2026.

KABUM. **Impressora 3D Creality K1 Max**. Disponível em:

<https://www.kabum.com.br/produto/495704/impressora-3d-creality-k1-max-velocidade-maxima->

600mm-s-1202080002. Acesso em: 27 set. 2026.

LC FERRAGENS. **Multímetro digital Hikari HM-1000**. Disponível em: <https://www.lcferragens.com.br/produto/multimetro-hikari-digital-hm-1000/>. Acesso em: 27 set. 2026.

LEROY MERLIN. **Filamento PLA premium 1 kg Voolt3D**. Disponível em: https://www.leroymerlin.com.br/filamento-pla-preto-premium-1kg-voolt3d-oficial_1570566287. Acesso em: 27 set. 2026.

LOJA DO MECÂNICO. **Multímetro digital HM-1001 Hikari**. Disponível em: <https://www.lojadomecanico.com.br/produto/123563/3/47/multimetro-digital-hm-1001-hikari-21n240>. Acesso em: 27 set. 2026.

MAGAZINE LUIZA. **Fonte 5 V 3 A com saída USB-C**. Disponível em: <https://www.magazineluiza.com.br>. Acesso em: 27 set. 2026.

MAKERHERO. **Capa para chave tátil push button 12 x 12 mm**. Disponível em: <https://www.makehero.com/produto/capa-para-chave-tactil-push-button-12x12-mm/>. Acesso em: 27 set. 2026.

MAKERHERO. **Fonte DC chaveada 5 V 3 A USB tipo C**. Disponível em: <https://www.makehero.com/produto/fonte-dc-chaveada-5v-3a-usb-tipo-c/>. Acesso em: 27 set. 2026.

MAMUTE ELETRÔNICA. **Conector jack USB tipo C fêmea com rabicho de 2 fios**. Disponível em: <https://www.mamuteeletronica.com.br/conector-jack-usb-tipo-c-femea-com-rabicho-2-fios-22739>. Acesso em: 27 set. 2026.

MERCADO LIVRE. **Bateria 18650 3,7 V 3.800 mAh Onistek**. Disponível em: <https://www.mercadolivre.com.br>. Acesso em: 27 set. 2026.

MERCADO LIVRE. **Módulo RTC (real time clock) DS3231**. Disponível em: <https://www.mercadolivre.com.br/modulo-rtc-real-time-clock-ds3231-arduino-esp8266-esp32-rasp/p/MLB42966007>. Acesso em: 27 set. 2026.

MERCADO LIVRE. **Módulo de carga e descarga para bateria 18650 IP5306, 2 A, 5 V, USB-C**. Disponível em: <https://www.mercadolivre.com.br/modulo-carga-y-descarga-bateria-18650-ip5306-2a-5v-usb-c/p/MLB2064132227>. Acesso em: 27 set. 2026.

MERCADO LIVRE. **Suporte para 1 bateria 18650**. Disponível em: <https://www.mercadolivre.com.br>. Acesso em: 27 set. 2026.

MERCADO SHOPS. **Kit com 5 conectores jack tipo C fêmea com rabicho de 2 fios**. Disponível em: <https://ralphcouch.mercadoshops.com.br/MLB-3667070205-kit-com-5-conector-jack-tipo-c-com-rabicho-2-fios-fmea-JM>. Acesso em: 27 set. 2026.

METALÚRGICA FERRI. **Quanto custa um molde de injeção plástica: como o orçamento é formado**. Disponível em: <https://www.metalferri.com.br/blog/quanto-custa-molde-injecao-plastica>. Acesso em: 27 set. 2026.

MIL PLÁSTICOS. **ABS granulado: preço**. Disponível em: <https://www.milplasticos.com.br/abs-granulado-preco>. Acesso em: 27 set. 2026.

MULTCOMERCIAL. **Cabinho flexível 0,32 mm² (22 AWG)**. Disponível em: <https://www.multcomercial.com.br/fios-e-cabos/cabinho-flexivel/flex-0-32mm-22-awg.html>. Acesso em: 27 set. 2026.

NOVA TRIDA ELETRÔNICA. **Capacitor eletrolítico 1000 µF x 16 V**. Disponível em: <https://www.novatridaeletronica.com.br/capacitor-eletrolitico-1000uf-x-16v>. Acesso em: 27 set. 2026.

NTC. **Custo de molde de injeção de plástico**. Disponível em: <https://ntc.ind.br/custo-de-molde-injecao-plastico/>. Acesso em: 27 set. 2026.

OCTET3D. **Quanto custa imprimir em 3D? Guia de preços 2026**. Disponível em: <https://octet3d.com/blog/quanto-custa-imprimir-em-3d>. Acesso em: 27 set. 2026.

ORIELEC. **Bateria de lítio LIR2032**. Disponível em: <https://www.lojaorielec.com.br/acessorios/baterias-e-suportes/bateria-de-litio-lir2032>. Acesso em: 27 set. 2026.

PV MAGAZINE BRASIL. **Aneel projeta aumento tarifário médio de 8% em 2026.** Disponível em: <https://www.pv-magazine-brasil.com/2026/03/17/aneel-projeta-aumento-tarifario-medio-de-8-em-2026/>. Acesso em: 27 set. 2026.

REVISTA FÓRUM. **Preço do dólar hoje: veja a cotação da moeda em reais atualizada.** Disponível em: <https://revistaforum.com.br/economia/preco-dolar-24-09-2026/>. Acesso em: 27 set. 2026.

SMARTPROJECTS. **Placa fenolite perfuradailhada, 7 × 9 cm.** Disponível em: <https://www.smartprojectsbrasil.com.br/placa-fenolite-perfurada-ilhada-fibra-de-vidro-7x9-cm>. Acesso em: 27 set. 2026.

SUBMARINO. **Cabo micro-USB, 1 metro.** Disponível em: <https://www.submarino.com.br>. Acesso em: 27 set. 2026.

TERMOTUBOS. **Kit de termo-retráteis variados.** Disponível em: <https://loja.termotubos.com.br/kits/termo-retrateis-variados>. Acesso em: 27 set. 2026.

USINAINFO. **Conector USB-C fêmea com rabicho de alimentação de 2 fios para painel.** Disponível em: <https://www.usinainfo.com.br/conector-usb/conector-usb-c-femea-com-rabicho-de-alimentacao-2-fios-para-painel-diy-9169.html>. Acesso em: 27 set. 2026.

VAMUINO. **Buzzer ativo 5 V.** Disponível em: <https://lojinha.vamuino.com.br/produto/buzzer-ativo-5v/>. Acesso em: 27 set. 2026.

WEMAK. **Filamento PLA premium 1 kg Voolt3D.** Disponível em: <https://www.wemak.com.br/produtos/filamento-3d-pla-1kg-impressao-3d-premium-1-75mm-voolt3d-cor-preto/>. Acesso em: 27 set. 2026.

WJ COMPONENTES. **Micro servo SG90 9 g.** Disponível em: <https://www.wjcomponentes.com.br/micro-servo-sg90-9g/>. Acesso em: 27 set. 2026.